



Écrit par L'architech

# INTRODUCTION

TorusNet est une **architecture directionnelle souveraine**, conçue pour établir des communications sécurisées sans recourir aux mécanismes cryptographiques traditionnels. Contrairement aux protocoles classiques fondés sur des structures mathématiques dérivables, des clés privées ou des certificats, TorusNet repose sur une **géométrie interne vivante**, formée de tores directionnels dont la structure évolue à chaque clé.

La sécurité n'est plus assurée par des opérations mathématiques, mais par :

- l'inversion directionnelle des bits normal/fantôme,
- la mutation dynamique des IDs internes,
- la synchronisation Host–Client–Host,
- la cascade orbitale multi-paliers,
- la physique directionnelle du tore privé,
- la quadruple-identité matérielle,
- et la destruction directionnelle en fin de session.

Ces mécanismes rendent le tore privé **non transmissible, non dérivable, non clonable et impossible à simuler**, même par des systèmes quantiques ou photoniques.

## 1. Les trois tores fondamentaux

L'architecture TorusNet repose sur trois structures directionnelles :

1. **Le Tore Public** Espace de lecture directionnelle utilisé pour les déplacements exponentiels issus de la cascade orbitale.
2. **Le Tore d'Authentification** Transmis une seule fois au début de la session, il sert de point d'ancrage directionnel initial.
3. **Le Tore Privé** Reconstitue localement, jamais transmis, vivant, oscillant, auto-modifiant, et impossible à dériver.

Le Tore Privé possède un **ID unique**, encodé dans la distribution fractale de ses IDs internes. Cet ID structurel permet au Host de déclencher un **renouvellement directionnel** via une clé spéciale.

## 2. La clé directionnelle : 64 bits, dont 30 bits orbitaux

Chaque clé directionnelle fait **64 bits**. Elle contient :

- des informations de type (00, 01, 10, 11),
- des identifiants directionnels,
- des paramètres de session,
- **et un segment de 30 bits dédié aux déplacements directionnels.**

Ce segment de 30 bits n'est pas traité comme une suite linéaire : il est transformé en **anneau circulaire de 30 positions**, générant :

- 30 lectures directes,
- 30 lectures inverses,
- soit **60 états orbitaux**,
- produisant **1800 bits**,
- découpés en **180 blocs directionnels (N,E)**.

Ces déplacements lisent ensuite :

- **5400 bits** dans le tore public (palier 2),
- puis **16200 bits**,
- puis **48600 bits**,
- jusqu'à **145800 bits** (palier 5).

La cascade est **bijective**, totalement réversible, et permet de reconstruire le tore privé ou la clé à partir de l'un ou l'autre.

### 3. Séquence directionnelle Host–Client–Host

La transmission d'une clé directionnelle suit **exactement cinq étapes**, chacune assurant une synchronisation parfaite entre le Tore Privé du Host et celui du Client. Cette séquence garantit qu'aucune divergence directionnelle n'est possible.

#### 3.1. Host → Client : Transmission initiale de la clé

Le Host envoie la clé directionnelle brute au Client. Cette clé contient :

- le type (00, 01, 10, 11),
- l'ID de case ciblé,
- les 30 bits de déplacement fractal,
- les informations directionnelles nécessaires au parcours.

Le Client reçoit la clé mais **ne l'exécute pas encore**.

#### 3.2. Client → Host : Renvoi de la clé pour validation

Le Client renvoie **exactement la même clé** au Host.

Le Host :

- compare la clé envoyée et la clé reçue,
- vérifie qu'aucune altération n'a eu lieu,
- confirme que le Client est bien l'émetteur légitime,
- inscrit l'IP du Client comme **IP de confiance** pour cette session.

Si la clé diffère, la session est immédiatement détruite.

### 3.3. Host → Client : Renvoi de la copie validée

Le Host renvoie une seconde copie de la clé, cette fois **validée**.

Le Client :

- compare la clé validée avec celle qu'il a renvoyée,
- confirme la cohérence,
- obtient l'autorisation d'exécuter le parcours directionnel.

À partir de ce moment, la clé est considérée comme **authenticifiée**.

### 3.4. Client → Host : Renvoi de la clé après exécution

Le Client exécute la clé dans son Tore Privé :

- lecture des blocs de 3 bits du Tore Public,
- sélection des chemins internes,
- inversions normal/fantôme,
- mutation de l'ID de case (premiers 16 bits),
- mutation de l'ID global (seconds 16 bits),
- progression fractale complète.

Une fois l'exécution terminée :

**Le Client renvoie la clé au Host pour indiquer que son tore a été mis à jour.**

Ce renvoi est la **preuve directionnelle** que le parcours a été exécuté.

### 3.5. Host → Client : Confirmation finale et synchronisation

Le Host exécute à son tour la clé dans son propre Tore Privé :

- il lit les mêmes blocs,
- suit les mêmes chemins internes,
- effectue les mêmes inversions,
- reconstruit les mêmes IDs,
- met à jour son tore de manière identique.

Une fois son tore modifié :

**Le Host renvoie la clé au Client pour confirmer la réception ET confirmer que son tore a bien été modifié.**

Cette étape finalise la synchronisation.

À la fin de cette étape :

**Les deux tores privés sont parfaitement synchronisés. Aucune divergence n'est structurellement possible.**

## **4. Physique directionnelle et cycle de vie du tore**

Chaque case du tore privé contient :

- un ID directionnel,
- un bit normal,
- un bit fantôme,
- quatre directions cardinales/diagonales,
- des déplacements exponentiels NE.

Avant lecture, les bits normal/fantôme sont en **superposition directionnelle**. La lecture effondre l'état, comme un spin quantique.

Le tore privé :

- vit en RAM,
- n'est jamais transmis,
- est chiffré en fin de session,
- renaît à chaque permutation,
- est détruit structurellement en cas d'anomalie.

## **5. Quadruple-identité matérielle**

La reconnexion nécessite la correspondance exacte de :

- l'adresse MAC,
- l'ID BIOS,
- l'ID CPU,
- l'ID interne du tore privé.

Cette quadruple-identité rend le tore :

- intransférable,
- non clonable,
- non utilisable hors de son matériel d'origine,
- impossible à simuler ou virtualiser.

## 6. HTTPS et DNS : mécanismes temporaires

TorusNet utilise encore :

- **HTTPS** pour la première clé,
- **DNS** comme filtre préliminaire.

Ces mécanismes sont **temporaires** et seront remplacés par :

- un **DNS directionnel**,
- une **validation directionnelle pré-session**,
- une **identité directionnelle native**.

## 7. Politique de diffusion : délai d'un an

Un délai officiel de **12 mois** est accordé aux institutions contactées pour :

- analyser le protocole,
- demander des informations,
- proposer une collaboration.

Sans réponse après un an, TorusNet et Mondus sont **publiés en open-source**.

## 8. Mondus : le réseau post-Internet

Mondus n'est pas une évolution de l'Internet. C'est un **réseau directionnel post-Internet**, fondé sur :

- des tores vivants,
- une géométrie directionnelle,
- une sécurité structurelle,
- une identité matérielle,
- une communication non transmissive.

Mondus représente la transition vers un futur où les réseaux ne transmettent plus des données, mais des **directions**, et où la géométrie devient le fondement de la sécurité numérique.

# Chapitre 1 — Structure des Tores Directionnels

## 1.1. Vue d'ensemble

L'architecture de TorusNet repose sur **trois tores directionnels vivants**, chacun possédant une géométrie interne propre et jouant un rôle essentiel dans la cohérence structurelle de la session. Ces tores ne sont pas des objets mathématiques classiques : ce sont des **espaces directionnels fermés**,

non dérivables, non transmissibles, et interprétables uniquement par la **mécanique fractale, orbitale et directionnelle** du protocole.

Les trois tores sont :

1. **Le Tore Public**
2. **Le Tore d'Authentification**
3. **Le Tore Privé**

Chaque tore possède une structure interne spécifique et un rôle directionnel précis dans la session.

## **1.2. Le Tore Public**

### **1.2.1. Définition**

Le Tore Public est un espace directionnel **exhaustif**, contenant **toutes les combinaisons possibles des 30 bits**. Il ne s'agit plus d'un tore arbitraire : c'est un **réservoir directionnel complet**, couvrant l'intégralité des états possibles pour les blocs de 3 bits utilisés par la cascade orbitale.

Il ne contient aucune information sensible, aucune clé privée, aucune structure dérivable. Son rôle est strictement opérationnel et universel : **tous les clients utilisent le même Tore Public**.

### **1.2.2. Taille et structure interne**

Le Tore Public contient :

$2^{30}=1\,073\,741\,824$  cases

Chaque case contient **exactement 30 bits**, représentant une combinaison unique parmi les 1 073 741 824 états possibles.

Ces 30 bits servent à :

- la lecture orbitale,
- la progression fractale,
- la cohérence interne du Tore Privé,
- la stabilité directionnelle du protocole.

Contrairement aux autres tores :

- **aucune paire normal/fantôme,**
- **aucun chemin interne,**
- **aucun ID directionnel,** ne sont présents dans le Tore Public.

Il est un **réservoir directionnel universel**, identique pour tous les utilisateurs.

### **1.2.3. Rôle dans la session**

Le Tore Public :

- fournit les blocs de 30 bits aux déplacements exponentiels,
- sert de base directionnelle stable pour la cascade orbitale,

- influence la reconstruction du Tore Privé,
- ne contient aucune donnée exploitable par un attaquant.

## 1.3. Le Tore d'Authentification

### 1.3.1. Définition

Le Tore d'Authentification est transmis **une seule fois** au début de la session. Il sert à établir la cohérence initiale entre le Host et le Client.

### 1.3.2. Taille et structure interne

Le Tore d'Authentification est un tableau de **1000 × 1000 cases**, soit **1 000 000 cases**, contenant chacune **3 bits** :

- Bit 0 : normal (0) ou fantôme (1)
- Bits 1–2 : chemin interne (00, 01, 10, 11)

Il contient également :

- des chemins internes d'initialisation,
- des paires normal/fantôme cohérentes,
- des identifiants directionnels initiaux.

### 1.3.3. Rôle dans la session

Le Tore d'Authentification :

- valide la première clé directionnelle,
- établit la cohérence initiale,
- initialise l'ID global,
- prépare la reconstruction du Tore Privé.

Une fois cette étape terminée, il n'est plus utilisé.

## 1.4. Le Tore Privé

### 1.4.1. Définition

Le Tore Privé est l'élément central de TorusNet. Il est reconstruit localement, jamais transmis, et constitue un espace directionnel :

- vivant,
- oscillant,
- auto-modifiant,
- impossible à dériver,
- impossible à simuler.

### 1.4.2. Taille et structure interne

Le Tore Privé est un tableau de **1000 × 1000 cases**, soit **1 000 000 cases**, contenant chacune :

- un bit normal,
- un bit fantôme,
- un chemin interne (00, 01, 10, 11),
- un ID directionnel (16 bits),
- quatre directions cardinales/diagonales,
- des déplacements exponentiels NE.

Il possède également :

- un **ID unique du tore**, encodé dans la distribution fractale des IDs internes,
- un **ID global**, mis à jour à chaque clé.

Sa structure évolue à **chaque clé directionnelle validée**, via :

- l'inversion normal/fantôme,
- la mutation d'ID,
- la progression fractale,
- la synchronisation Host–Client–Host.

### 1.4.3. Reconstruction fractale et orbitale (corrigée)

Chaque clé directionnelle (64 bits) déclenche une reconstruction directionnelle fondée sur la **cascade orbitale**. La clé contient un segment de **30 bits de déplacement**, transformé en **anneau circulaire** permettant :

- **30 lectures directes**,
- **30 lectures inverses**,
- soit **60 états orbitaux**,
- produisant **1800 bits** (palier 1).

Ces 1800 bits sont découpés en **180 blocs directionnels (N,E)**, chacun définissant un déplacement exponentiel dans le tore public.

La reconstruction se fait ensuite par **paliers successifs**, chaque palier amplifiant la lecture :

| Palier | Bits générés | Déplacements | Description                               |
|--------|--------------|--------------|-------------------------------------------|
| 1      | 1800 bits    | 180          | Orbite circulaire (60 lectures × 30 bits) |
| 2      | 5400 bits    | 540          | Lecture du tore public ×30 bits           |
| 3      | 16200 bits   | 1620         | Amplification directionnelle              |

| Palier | Bits générés | Déplacements | Description                        |
|--------|--------------|--------------|------------------------------------|
| 4      | 48600 bits   | 4860         | Expansion fractale                 |
| 5      | 145800 bits  | 14580        | Projection directionnelle complète |

Le tore public est ainsi **lu 10 fois en cascade**, conformément à la physique directionnelle :

- 10 niveaux fixes,
- 10 lectures successives,
- 10 effondrements directionnels,
- garantissant la réversibilité et la stabilité du tore privé.

La reconstruction orbitale et fractale est :

- **non dérivable,**
- **non transmissible,**
- **non réversible,**
- **entièrement dépendante de l'état interne du tore privé,**
- **stabilisée par l'inversion normal/fantôme et la mutation d'ID.**

#### 1.4.4. Rôle dans la session

Le Tore Privé :

- valide les clés directionnelles,
- génère les IDs internes,
- assure la cohérence directionnelle,
- protège la session contre toute dérivabilité,
- constitue la base de la souveraineté structurelle.

Il est **détruit directionnellement** en fin de session ou en cas d'anomalie grave.

## 1.5. Relations entre les trois tores

### 1.5.1. Tore Public → Tore Privé

Le Tore Public fournit les blocs de 3 bits nécessaires à la cascade orbitale et à la reconstruction fractale du Tore Privé.

### 1.5.2. Tore d'Authentification → Tore Privé

Le Tore d'Authentification valide la première clé et initialise la cohérence directionnelle du Tore Privé.

### 1.5.3. Tore Privé → Session

Le Tore Privé détermine :

- la validité des clés,

- la progression fractale,
- la cohérence directionnelle,
- la sécurité structurelle.

## 1.6. Propriétés structurelles des tores

Les trois tores possèdent des propriétés communes :

- non transmissibles (sauf Tore Public + Tore d'Authentification),
- non dérivables,
- non réversibles,
- non exploitables,
- géométriques,
- directionnels,
- fractals,
- inviolables par conception.

Ces propriétés constituent la base de la sécurité post-quantique et post-photonique de TorusNet.

## 1.7. Résumé du Chapitre 1

Le Chapitre 1 établit la structure fondamentale de TorusNet :

- un Tore Public exhaustif de  $2^{30}$  cases  $\times$  30 bits,
- un Tore d'Authentification de  $1000 \times 1000$  cases  $\times$  3 bits,
- un Tore Privé vivant, oscillant, auto-modifiant,
- une géométrie interne non dérivable,
- une mécanique fractale et orbitale de reconstruction,
- une séparation stricte entre les rôles,
- une souveraineté structurelle totale.

Ces éléments forment le socle sur lequel reposent :

- les clés directionnelles,
- la transmission Host–Client–Host,
- la Forteresse de Verre,
- la synchronisation,
- le multi-tore,
- la sécurité directionnelle,
- et la destruction structurelle.

# Chapitre 2 — Structure des Clés Directionnelles

## 2.1. Vue d'ensemble

Les clés directionnelles constituent l'unité fondamentale de progression dans le protocole. Chaque clé est un paquet de **64 bits**, transmis selon une séquence directionnelle **bidirectionnelle et synchronisée** entre le Host et le Client.

Une clé directionnelle n'est **pas** une commande cryptographique : elle sert uniquement de **vecteur directionnel**, déclenchant la mécanique orbitale et fractale interne qui modifie le Tore Privé.

Une clé directionnelle :

- ne contient aucune information sensible,
- ne peut pas être dérivée,
- ne peut pas être inversée,
- ne peut pas être exploitée isolément,
- n'a de sens que dans le contexte du tore privé et de la cascade directionnelle.

## 2.2. Découpage officiel des 64 bits

Le découpage interne d'une clé directionnelle est strictement défini. Il ne s'agit pas d'un format cryptographique, mais d'une **structure directionnelle** permettant :

- la navigation dans le Tore Public,
- la reconstruction du Tore Privé,
- la définition du type de clé,
- la validation directionnelle interne,
- la mutation d'ID,
- la synchronisation Host–Client–Host.

### 2.2.1. Tableau de découpage des 64 bits

| Segment                | Bits  | Taille  | Description                                                                      |
|------------------------|-------|---------|----------------------------------------------------------------------------------|
| Point de départ        | 0–7   | 8 bits  | 4 bits pour N, 4 bits pour E → déplacement exponentiel $N^E$                     |
| Point d'arrivée        | 8–15  | 8 bits  | Même logique : $N^E$                                                             |
| Type de clé            | 16–17 | 2 bits  | 00 = reconstruction ; 11 = reconstruction de clés ; 01 = clé suivie ; 10 = arrêt |
| Navigation Tore Public | 18–47 | 30 bits | Segment orbital circulaire (anneau)                                              |
| ID de la case visée    | 48–63 | 16 bits | Identifiant interne de la case du Tore Privé                                     |

## 2.3. Interprétation des segments

### 2.3.1. Point de départ (8 bits)

Les 8 bits sont divisés en :

- 4 bits pour un entier N,
- 4 bits pour un exposant E,

formant un déplacement exponentiel :

$$\Delta = N^E$$

Ce déplacement détermine la **première case** à lire dans le Tore Public.

### 2.3.2. Point d'arrivée (8 bits)

Même structure :

- 4 bits pour N,
- 4 bits pour E,

formant :

$$\Delta = N^E$$

Ce déplacement détermine la **dernière case** à lire dans le Tore Public.

Les deux ensemble définissent une **fenêtre directionnelle**.

### 2.3.3. Type de clé (2 bits)

| Bits | Type                          | Description                                         |
|------|-------------------------------|-----------------------------------------------------|
| 00   | Clé de reconstruction         | Lecture directionnelle pour reconstruire un fichier |
| 11   | Clé de reconstruction de clés | Sert à reconstruire d'autres clés                   |
| 01   | Clé suivie                    | Une autre clé suit immédiatement                    |
| 10   | Clé d'arrêt                   | Termine la séquence                                 |

Ces types définissent la **logique directionnelle de session**, pas des commandes cryptographiques.

### 2.3.4. Navigation dans le Tore Public (30 bits)

Les **30 bits** constituent le **segment orbital** de la clé.

Ils sont transformés en :

- un **anneau circulaire de 30 positions**,
- 30 lectures directes,
- 30 lectures inverses,
- soit **60 états orbitaux**,
- produisant **1800 bits** (palier 1).

Ces 1800 bits sont découpés en **180 blocs directionnels (N,E)**, chacun définissant un déplacement exponentiel dans le Tore Public.

Chaque déplacement lit **un bloc de 3 bits** dans le Tore Public.

Les 30 bits ne sont jamais interprétés comme des commandes : ils sont une **structure géométrique orbitale**.

### 2.3.5. ID de la case visée (16 bits)

Les 16 bits représentent l'**identifiant interne** de la case du Tore Privé ciblée par la clé.

Cet ID :

- valide la cohérence fractale,
- permet de vérifier que la case ciblée est correcte,
- empêche toute falsification directionnelle,
- participe à la mutation directionnelle de l'ID de départ (MAJ 1).

Il s'agit d'un **identifiant géométrique**, non cryptographique.

## 2.4. Rôle directionnel des clés

### 2.4.1. Déclenchement de la cascade orbitale

Chaque clé déclenche :

- la génération de l'anneau de 30 bits,
- 60 lectures orbitale,
- 180 blocs directionnels (N,E),
- la lecture du Tore Public en **10 paliers successifs**,
- l'inversion normal/fantôme,
- la mutation d'ID,
- la mise à jour du Tore Privé.

### 2.4.2. Reconstruction du Tore Privé

Les clés directionnelles :

- reconstruisent le Tore Privé,
- mettent à jour les paires normal/fantôme,
- mutent les IDs internes,
- stabilisent l'ID global,
- valident les chemins internes,
- synchronisent Host et Client.

### 2.4.3. Absence de dérivabilité

Une clé directionnelle :

- ne peut pas être prédite,
- ne peut pas être reconstruite,
- ne peut pas être inversée,
- ne peut pas être analysée isolément,
- n'a de sens que dans le contexte du tore privé.

## 2.5. Transmission directionnelle

La transmission suit désormais **six étapes** (MAJ 1), garantissant une synchronisation parfaite :

1. Host → Client : envoi de la clé
2. Client → Host : renvoi de la clé
3. Host → Client : renvoi de la copie validée
4. Client : exécution du parcours directionnel
5. Client → Host : confirmation
6. Host : exécution du même parcours

Les deux tores privés évoluent **de manière symétrique**.

## 2.6. Gestion des pertes de clé

Si aucune réponse après 5 secondes :

- le Host génère une **nouvelle clé**,
- avec un **point de départ différent**,
- mais reconstruisant **le même fichier directionnel**.

Le protocole **ne retransmet jamais** le même paquet : il **régénère** une clé cohérente.

## 2.7. Résumé du Chapitre 2

Ce chapitre établit :

- le découpage officiel des 64 bits,
- la logique orbitale et directionnelle interne,
- la transmission en six étapes,
- la régénération directionnelle en cas de perte,
- l'impossibilité de dérivation,
- le rôle des paliers dans la reconstruction du tore privé.

Les clés directionnelles sont le **moteur de la progression orbitale et fractale**.

# Chapitre 3 — Mécanique Fractale et Blocs de 3 Bits

## 3.1. Vue d'ensemble

La mécanique fractale constitue le cœur opérationnel du protocole. Elle est responsable de :

- la lecture des **blocs de 3 bits** dans le Tore Public,
- la progression directionnelle interne,
- la reconstruction du Tore Privé,
- la création et mutation des identifiants internes,
- la validation des chemins internes,
- la cohérence structurelle de la session.

Chaque clé directionnelle déclenche une **cascade orbitale**, composée de plusieurs paliers successifs qui amplifient la lecture directionnelle et modifient le Tore Privé.

La mécanique fractale est :

- directionnelle,
- non linéaire,
- non dérivable,
- dépendante de l'état interne du tore,
- stabilisée par l'inversion normal/fantôme et la mutation d'ID.

## 3.2. Les blocs de 3 bits : structure fondamentale

Les blocs de **3 bits** sont la plus petite unité directionnelle du protocole. Ils sont présents :

- dans le Tore Public (extraits des 30 bits),
- dans le Tore d'Authentification,
- dans le Tore Privé.

Chaque bloc de 3 bits est interprété selon la règle géométrique suivante :

| Bit | Signification |
|-----|---------------|
|-----|---------------|

|       |                           |
|-------|---------------------------|
| Bit 0 | Normal (0) ou Fantôme (1) |
|-------|---------------------------|

|          |                                 |
|----------|---------------------------------|
| Bits 1–2 | Chemin interne (00, 01, 10, 11) |
|----------|---------------------------------|

Cette règle est **inviolable**. Les blocs de 3 bits **ne codent aucune instruction** : ils définissent uniquement une **direction interne**.

## 3.3. Interprétation du bit normal/fantôme

Le premier bit du bloc détermine la nature du déplacement, mais dans le protocole moderne, il possède une propriété essentielle :

✓ **Le bit normal et le bit fantôme sont inversés simultanément à chaque clé validée.**

Cette inversion simultanée est une règle structurelle introduite dans la mécanique directionnelle (MAJ 1). Elle transforme le tore privé en un espace oscillant, vivant et anti-prédictif.

### 3.3.1. Bit normal (0)

Un bit normal indique :

- un déplacement cohérent,
- une progression stable,
- une lecture directe du chemin interne.

Lors de la validation d'une clé :

- **normal = 0 → 1**
- cette inversion modifie la géométrie interne du tore
- elle participe à la mutation directionnelle de la case

### 3.3.2. Bit fantôme (1)

Un bit fantôme indique :

- un déplacement inversé,
- une progression indirecte,
- une lecture inversée du chemin interne.

Lors de la validation d'une clé :

- **fantôme = 1 → 0**
- l'inversion simultanée normal/fantôme crée une oscillation directionnelle
- elle empêche toute dérivabilité ou simulation du tore

### 3.3.3. Inversion multiple

Si le parcours repasse sur une case :

- l'inversion se produit **à nouveau**,
- créant une oscillation directionnelle multiple,
- rendant le tore encore plus imprévisible.

## 3.4. Les quatre chemins internes

Les bits 1 et 2 définissent l'un des quatre chemins internes :

| Bits | Chemin   | Description          |
|------|----------|----------------------|
| 00   | Chemin A | Déplacement linéaire |
| 01   | Chemin B | Déplacement diagonal |

| Bits | Chemin   | Description             |
|------|----------|-------------------------|
| 10   | Chemin C | Déplacement exponentiel |
| 11   | Chemin D | Déplacement inversé     |

Ces chemins ne sont pas des commandes : ils définissent la **géométrie interne du tore**.

### 3.5. Lecture fractale dans le Tore Public

Les **30 bits orbitaux** de la clé directionnelle sont transformés en :

- **30 lectures directes**,
- **30 lectures inverses**,
- soit **60 états orbitaux**,
- produisant **1800 bits** (palier 1).

Ces 1800 bits sont découpés en **180 blocs directionnels (N,E)**. Chaque bloc (N,E) définit un déplacement exponentiel dans le Tore Public.

Chaque déplacement lit **un bloc de 3 bits** dans le Tore Public.

La cascade se poursuit ensuite sur **10 paliers successifs**, chacun amplifiant la lecture :

| Palier | Bits générés               | Déplacements             |
|--------|----------------------------|--------------------------|
| 1      | 1800 bits                  | 180                      |
| 2      | 5400 bits                  | 540                      |
| 3      | 16200 bits                 | 1620                     |
| 4      | 48600 bits                 | 4860                     |
| 5      | 145800 bits                | 14580                    |
| 6–10   | Amplifications successives | Expansion directionnelle |

La lecture fractale est :

- directionnelle,
- non linéaire,
- non dérivable,
- dépendante de l'état interne du tore.

### 3.6. Progression fractale interne

La progression fractale suit trois règles fondamentales :

#### 3.6.1. Règle de cohérence

Un déplacement est valide si :

- le bit normal/fantôme est cohérent avec l'état de la case,
- le chemin interne est compatible avec la progression précédente.

#### 3.6.2. Règle de continuité

La progression doit :

- rester dans les limites du tore,
- suivre les chemins internes,
- respecter les inversions fantômes.

### 3.6.3. Règle d'expansion

Les déplacements exponentiels (chemin C) :

- augmentent la vitesse de progression,
- permettent de traverser rapidement le tore,
- créent et mutent les IDs internes.

## 3.7. Mutation directionnelle de l'ID de la case de départ (MAJ 1)

Lorsqu'une clé parcourt le Tore Privé :

✓ **L'ID de la case de départ mute automatiquement vers un nouvel ID directionnel.**

Cette mutation :

- dépend des premiers bits de la clé,
- modifie l'identité directionnelle de la case,
- change la géométrie interne du tore,
- rend les IDs dépendants de l'historique des clés,
- empêche toute dérivabilité ou reconstruction externe.

La mutation d'ID est un mécanisme central du tore privé.

## 3.8. Cas particulier : destruction directionnelle

La destruction directionnelle intervient :

- en fin de session,
- en cas d'anomalie grave,
- en cas de clé d'arrêt (type 10).

Elle consiste à :

- rendre incohérentes les paires normal/fantôme,
- invalider les chemins internes,
- effacer les IDs internes,
- effacer l'ID global,
- effondrer la géométrie directionnelle.

La destruction n'est **pas aléatoire** : elle est **structurellement incohérente**, empêchant toute analyse post-mortem, même par des systèmes quantiques ou photoniques.

## 3.9. Résumé du Chapitre 3

Ce chapitre établit :

- la structure des blocs de 3 bits,
- la logique normal/fantôme et leur inversion simultanée,
- les quatre chemins internes,
- la lecture orbitale dans le Tore Public,
- la cascade directionnelle par paliers,
- la mutation directionnelle des IDs internes,
- la progression directionnelle,
- la destruction directionnelle.

La mécanique fractale est le **moteur interne du tore privé**. Elle garantit une progression :

- non dérivable,
- non réversible,
- impossible à simuler,
- inviolable par conception.

# Chapitre 4 — La Forteresse de Verre (Pare-feu Directionnel)

## 4.1. Vue d'ensemble

La Forteresse de Verre est le **pare-feu structurel directionnel** du protocole. Contrairement aux pare-feux traditionnels fondés sur des règles, des signatures, des ports ou des heuristiques, la Forteresse de Verre repose sur une **géométrie interne vivante**, inscrite dans :

- le Tore Public,
- le Tore d'Authentification,
- le Tore Privé,
- la mécanique fractale,
- la logique orbitale,
- la progression directionnelle Host → Client → Host.

Elle ne filtre pas les paquets selon des critères réseau classiques. Elle valide la **cohérence directionnelle** des clés, des déplacements, des IDs, des chemins internes et des inversions normal/fantôme.

La Forteresse de Verre constitue une barrière structurelle **impossible à contourner**, même pour un attaquant disposant d'un ordinateur quantique ou photonique.

## 4.2. Nature structurelle de la Forteresse de Verre

La Forteresse de Verre n'est :

- ni un logiciel,
- ni un module externe,
- ni un composant réseau.

C'est une **structure interne**, intégrée :

- au Tore Public,
- au Tore d'Authentification,
- au Tore Privé,
- à la mécanique fractale,
- à la cascade orbitale,
- à la logique directionnelle.

Elle fonctionne **sans** :

- signatures,
- règles,
- ports TCP,
- adresses MAC,
- horodatages,
- numéros de séquence,
- cryptographie.

Elle repose uniquement sur :

- la cohérence fractale,
- la géométrie interne,
- les paires normal/fantôme,
- les chemins internes,
- les IDs directionnels,
- l'ID unique du tore,

- la mutation d'ID,
- la progression Host → Client → Host.

### 4.3. Rôle de la Forteresse de Verre

La Forteresse de Verre assure quatre fonctions essentielles :

1. **Validation directionnelle des clés**
2. **Protection structurelle contre les intrusions**
3. **Surveillance des IP et des ports directionnels**
4. **Destruction immédiate en cas d'anomalie**

Ces fonctions ne sont pas exécutées par des règles logicielles, mais par la **structure interne du tore**.

### 4.4. Validation directionnelle des clés

La Forteresse de Verre valide chaque clé directionnelle selon :

- son type (00, 01, 10, 11),
- son point de départ ( $N^E$ ),
- son point d'arrivée ( $N^E$ ),
- ses **60 lectures orbitale**,
- ses **180 déplacements exponentiels**,
- les blocs de **3 bits** lus dans le Tore Public,
- l'ID de la case visée,
- la cohérence fractale interne,
- l'inversion normal/fantôme,
- la mutation d'ID de la case de départ.

Une clé est acceptée si :

- les déplacements sont cohérents,
- les paires normal/fantôme sont valides,
- les chemins internes sont compatibles,
- l'ID visé correspond à une case valide du Tore Privé,
- la mutation d'ID est légitime,
- la progression orbitale est cohérente.

Une clé est rejetée si :

- un déplacement est incohérent,

- un chemin interne est impossible,
- un ID ne correspond à aucune case,
- une inversion normal/fantôme est illégitime,
- une mutation d'ID est incohérente.

Le rejet est **structurel**, non logique.

## 4.5. Protection structurelle contre les intrusions

La Forteresse de Verre protège contre :

- les paquets falsifiés,
- les clés malformées,
- les clés incohérentes,
- les clés dérivées,
- les clés injectées,
- les clés répétées,
- les clés hostiles.

Elle ne compare pas les paquets à des signatures. Elle vérifie la **cohérence directionnelle**.

Un attaquant ne peut pas :

- dériver une clé,
- prédire une clé,
- reconstruire une clé,
- falsifier une clé,
- injecter une clé valide.

Toute clé hostile est immédiatement rejetée.

## 4.6. Surveillance des IP et des ports directionnels

La Forteresse de Verre surveille :

- l'IP du client,
- l'IP du host,
- les ports directionnels,
- les chemins internes,
- les IDs globaux,
- l'ID unique du tore.

### 4.6.1. IP de confiance

Lors de la première clé validée :

- l'IP du client est inscrite comme **IP de confiance**,
- toute autre IP est rejetée structurellement.

### 4.6.2. Ports directionnels

Les ports directionnels ne sont pas des ports TCP classiques. Ils sont définis par :

- les chemins internes,
- les paires normal/fantôme,
- les IDs internes,
- la géométrie directionnelle.

Un port directionnel :

- ne peut pas être scanné,
- n'existe pas dans la couche réseau classique,
- ne peut pas être falsifié.

## 4.7. Détection des anomalies directionnelles

La Forteresse de Verre détecte les anomalies suivantes :

- incohérence normal/fantôme,
- incohérence de chemin interne,
- incohérence d'ID,
- incohérence exponentielle,
- incohérence de progression fractale,
- incohérence de mutation d'ID,
- incohérence de transmission (Host → Client → Host).

### 4.7.1. Incohérence de transmission

Si la séquence :

1. Host → Client
2. Client → Host
3. Host → Client

n'est pas respectée, la clé est rejetée.

### 4.7.2. Incohérence fractale

Si une action fractale :

- lit un bloc impossible,
- atteint une case incohérente,
- crée un ID illégitime,
- suit un chemin interdit,
- produit une mutation d'ID incohérente,

la session est immédiatement détruite.

## 4.8. Destruction directionnelle en cas d'anomalie

La Forteresse de Verre déclenche la destruction directionnelle si :

- une clé est incohérente,
- une progression fractale est impossible,
- un ID est illégitime,
- une mutation d'ID est incohérente,
- un port directionnel est violé,
- une IP hostile est détectée.

La destruction directionnelle consiste à :

- rendre incohérentes les paires normal/fantôme,
- invalider les chemins internes,
- effacer les IDs internes,
- effacer l'ID global,
- effacer le Tore Privé.

Cette destruction n'est pas aléatoire : elle est **structurellement incohérente**, empêchant toute analyse post-mortem.

## 4.9. Résumé du Chapitre 4

La Forteresse de Verre est un **pare-feu structurel directionnel**. Elle :

- valide les clés,
- protège contre les intrusions,
- surveille les IP et les ports directionnels,
- détecte les anomalies fractales,
- détruit la session en cas d'incohérence,
- repose uniquement sur la géométrie directionnelle et la cohérence fractale.

Elle constitue la barrière structurelle fondamentale du protocole.

# Chapitre 5 — Handshake Directionnel (Séquence Host → Client → Host → Client → Host)

## 5.1. Vue d'ensemble

Le handshake directionnel est le mécanisme fondamental qui permet d'établir une session cohérente entre un host et un client. Contrairement aux protocoles classiques (TLS, SSH, IPSec) qui reposent sur :

- des clés privées,
- des certificats,
- des signatures,
- des dérivations mathématiques,
- des échanges cryptographiques,

le handshake directionnel repose exclusivement sur :

- la géométrie interne des tores,
- la cohérence fractale,
- la cascade orbitale,
- les déplacements directionnels,
- les IDs internes,
- la validation structurelle.

Il ne contient **aucune cryptographie**, aucune dérivation, aucune structure mathématique exploitable.

## 5.2. Objectifs du handshake

Le handshake directionnel assure :

1. la validation de la première clé directionnelle,
2. la synchronisation initiale entre host et client,
3. l'inscription de l'IP du client comme IP de confiance,
4. la création de l'ID global de session,
5. l'activation de la Forteresse de Verre,
6. la reconstruction initiale du Tore Privé,
7. la mise en place de l'ID unique du tore,
8. la synchronisation directionnelle complète via 5 transmissions.

Ces étapes sont **structurelles**, non cryptographiques.

### 5.3. La séquence officielle : 5 transmissions

La séquence directionnelle comporte **5 transmissions**, chacune ayant un rôle structurel précis. Elle est **obligatoire**, **inviolable**, et constitue la base de la synchronisation directionnelle.

## Transmission 1 — Host → Client

### Envoi initial de la clé directionnelle

Le host envoie une clé de 64 bits découpée en :

- 8 bits : point de départ ( $N^E$ ),
- 8 bits : point d'arrivée ( $N^E$ ),
- 2 bits : type de clé,
- 30 bits : anneau orbital,
- 16 bits : ID de la case visée.

Le client :

- reçoit la clé,
- vérifie sa structure,
- vérifie son type,
- vérifie l'ID visé,
- vérifie la cohérence fractale initiale.

Aucune lecture orbitale n'est encore exécutée.

## Transmission 2 — Client → Host

### Renvoi de la clé pour confirmer la réception

Le client renvoie **exactement la même clé** au host.

Le host :

- compare la clé envoyée et la clé reçue,
- valide la réception,
- inscrit l'IP du client comme **IP de confiance**,
- active la Forteresse de Verre.

Si la clé diffère → destruction immédiate.

## Transmission 3 — Host → Client

### Renvoi final de la clé pour autoriser l'exécution

Le host renvoie une **seconde copie** de la clé.

Le client :

- compare la clé renvoyée avec celle reçue,
- valide la cohérence,
- **exécute le parcours directionnel dans son tore privé**,
- inverse les bits normal/fantôme,
- **mute l'ID de la case de départ**,
- met à jour les chemins internes,
- complète la cascade orbitale (paliers 1 à 10).

Le tore privé du client devient **vivant, oscillant et modifié**.

## Transmission 4 — Client → Host

### Confirmation d'exécution du parcours

Le client renvoie la clé **après avoir exécuté le parcours**.

Ce renvoi indique au host :

- que les inversions normal/fantôme ont été faites,
- que l'ID de la case de départ a muté,
- que les chemins internes ont été mis à jour,
- que la progression fractale est cohérente,
- que le tore privé du client est à jour.

Le host peut alors :

- **exécuter le même parcours**,
- inverser normal/fantôme de son côté,
- **muter l'ID de la case de départ**,
- synchroniser son tore privé avec celui du client.

## Transmission 5 — Host → Client

### Confirmation finale de synchronisation

Le host renvoie la clé une dernière fois pour confirmer :

- qu'il a bien exécuté le parcours,
- qu'il a inversé normal/fantôme,
- qu'il a muté l'ID de la case de départ,
- que son tore privé est synchronisé avec celui du client,
- que la session est directionnellement stable.

Le client reçoit cette confirmation et la session devient **cohérente, stable et inviolable**.

## 5.4. Pourquoi cinq transmissions ?

### 5.4.1. Validation bidirectionnelle complète

Le host valide le client. Le client valide le host. Puis chacun valide l'exécution directionnelle de l'autre.

### 5.4.2. Protection contre les injections

Un attaquant ne peut pas :

- intercepter la clé,
- la modifier,
- la renvoyer,
- la falsifier.

Toute incohérence est détectée **structurellement**.

### 5.4.3. Protection contre les répétitions

Une clé répétée :

- est détectée,
- est rejetée,
- détruit la session.

### 5.4.4. Protection contre les dérivations

Un attaquant ne peut pas dériver :

- le point de départ,
- le point d'arrivée,
- les 30 bits orbitaux,
- les déplacements exponentiels,
- l'ID de la case visée,
- la mutation d'ID.

La structure directionnelle est **non dérivable**.

#### 5.4.5. Synchronisation directionnelle parfaite

Les 5 transmissions garantissent que :

- les deux tores privés ont exécuté le même parcours,
- les inversions normal/fantôme sont symétriques,
- les mutations d'ID sont identiques,
- la géométrie interne est cohérente,
- aucune divergence n'est possible.

### 5.5. Synchronisation initiale

Une fois la clé validée :

- le Tore d'Authentification est utilisé une seule fois,
- le Tore Privé est initialisé,
- l'ID global est créé,
- les paires normal/fantôme sont définies,
- les chemins internes sont activés,
- l'ID unique du tore est établi,
- la Forteresse de Verre est armée.

Cette synchronisation est **fractale**, non mathématique.

### 5.6. Reconstruction initiale du Tore Privé

La première clé déclenche :

- la génération de l'anneau orbital,
- les 60 lectures orbitale,
- les 180 blocs directionnels (N,E),
- la lecture des blocs de 3 bits dans le Tore Public,
- la mutation de l'ID de la case de départ,
- la mise à jour des chemins internes,
- la mise en place de la cohérence directionnelle.

Le Tore Privé devient **vivant, oscillant et opérationnel**.

### 5.7. Gestion des erreurs pendant le handshake

#### 5.7.1. Clé incohérente

Si la clé :

- contient un ID invalide,

- contient un point de départ impossible,
- contient un point d'arrivée impossible,
- contient un type illégitime,

la session est détruite.

### 5.7.2. Clé modifiée

Si la clé renvoyée par le client diffère :

- destruction immédiate,
- effacement du Tore Privé,
- invalidation de l'IP.

### 5.7.3. Absence de réponse

Si le host ne reçoit aucune réponse après 5 secondes :

- la clé est considérée perdue,
- une nouvelle clé est générée,
- avec un point de départ différent,
- mais reconstruisant **le même fichier directionnel**.

Aucune retransmission du même paquet n'est effectuée.

## 5.8. Résumé du Chapitre 5

Le handshake directionnel :

- repose sur **5 transmissions**,
- valide la première clé directionnelle,
- inscrit l'IP du client comme IP de confiance,
- initialise le Tore Privé,
- active la Forteresse de Verre,
- crée l'ID global,
- établit l'ID unique du tore,
- garantit une cohérence directionnelle totale,
- protège contre les injections, répétitions, dérivations et falsifications.

Il constitue la base de la **sécurité structurelle directionnelle**.

# Chapitre 6 — Synchronisation Directionnelle

## 6.1. Vue d'ensemble

La synchronisation directionnelle est le mécanisme qui garantit que :

- le Tore Public,
- le Tore d'Authentification,
- le Tore Privé,
- les clés directionnelles,
- la cascade orbitale,
- les IDs internes,
- les chemins directionnels,
- l'ID global,
- l'ID unique du tore,
- et la Forteresse de Verre

restent parfaitement alignés tout au long de la session.

Contrairement aux protocoles classiques qui synchronisent :

- des états cryptographiques,
- des horloges,
- des numéros de séquence,
- des compteurs,

la synchronisation directionnelle synchronise **des structures géométriques internes non dérivables**.

Elle est :

- **continue**,
- **fractale**,
- **bidirectionnelle**,
- **structurelle**,
- **vivante**,
- **oscillante** (inversion normal/fantôme),
- **mutante** (mutation d'ID).

## 6.2. Les trois tores et leurs rôles dans la synchronisation

La synchronisation repose sur l'interaction permanente entre :

### 6.2.1. Le Tore Public

- Source des blocs de 3 bits
- Base directionnelle universelle
- Contient toutes les combinaisons possibles des 30 bits
- Détermine les déplacements orbitaux et fractals
- Lu en **10 paliers successifs** à chaque clé

### 6.2.2. Le Tore d'Authentification

- Utilisé une seule fois
- Valide la première clé
- Initialise la cohérence directionnelle
- Définit les premières paires normal/fantôme
- Active les premiers chemins internes
- Crée l'ID global

### 6.2.3. Le Tore Privé

- Reconstitue localement
- Jamais transmis
- Contient 1 000 000 cases  $\times$  3 bits
- Évolue à chaque clé directionnelle
- Porte les IDs internes, l'ID global et l'ID unique du tore
- Subit l'inversion normal/fantôme et la mutation d'ID

La synchronisation consiste à maintenir une cohérence parfaite entre ces trois tores.

## 6.3. Déclenchement de la synchronisation

La synchronisation commence immédiatement après le **handshake directionnel à 5 transmissions** (Chapitre 5).

Une fois la première clé validée :

1. Le Tore d'Authentification est lu.
2. Le Tore Privé est initialisé.
3. Les premières paires normal/fantôme sont définies.
4. Les premiers chemins internes sont activés.
5. L'ID global est créé.
6. L'ID unique du tore est établi.

7. La Forteresse de Verre est armée.

8. Le Tore Public devient la source directionnelle principale.

À partir de ce moment, **chaque clé directionnelle maintient la synchronisation.**

## **6.4. Synchronisation fractale : principe général**

La synchronisation fractale repose sur trois règles fondamentales :

### **6.4.1. Règle de cohérence interne**

Chaque déplacement doit être cohérent avec :

- le bit normal/fantôme,
- le chemin interne,
- l'état de la case du Tore Privé,
- l'ID interne associé,
- l'ID global,
- l'ID unique du tore.

### **6.4.2. Règle de continuité directionnelle**

La progression doit :

- rester dans les limites du tore,
- suivre les chemins internes,
- respecter les inversions normal/fantôme,
- maintenir la cohérence des IDs internes,
- maintenir la cohérence de l'ID global.

### **6.4.3. Règle d'expansion fractale**

Les déplacements exponentiels (chemin C) :

- accélèrent la progression,
- créent des IDs internes,
- mutent les IDs existants,
- augmentent la complexité directionnelle,
- renforcent la cohérence fractale.

## **6.5. Synchronisation entre Tore Public et Tore Privé**

La synchronisation entre ces deux tores est la plus importante.

### **6.5.1. Lecture des blocs de 3 bits**

Les **30 bits orbitaux** de la clé directionnelle définissent :

- 60 lectures orbitale,
- 180 blocs directionnels (N,E),
- 180 déplacements exponentiels.

Chaque déplacement lit **un bloc de 3 bits** :

- normal/fantôme,
- chemin interne.

### 6.5.2. Mise à jour du Tore Privé

Chaque bloc lu :

- met à jour une case du Tore Privé,
- inverse normal/fantôme,
- active un chemin interne,
- crée ou valide un ID interne,
- **mute l'ID de la case de départ,**
- renforce l'ID unique du tore.

### 6.5.3. Cohérence directionnelle

La cohérence est assurée si :

- les déplacements sont compatibles,
- les chemins internes sont valides,
- les IDs internes sont cohérents,
- l'ID global reste stable,
- l'ID unique du tore reste cohérent.

## 6.6. Synchronisation entre Tore d'Authentification et Tore Privé

Cette synchronisation n'a lieu qu'une seule fois.

### 6.6.1. Validation initiale

Le Tore d'Authentification :

- valide la première clé,
- définit les premières paires normal/fantôme,
- active les premiers chemins internes,
- initialise l'ID global,
- initialise l'ID unique du tore.

### **6.6.2. Passage au Tore Public**

Une fois la première clé validée :

- le Tore d'Authentification n'est plus utilisé,
- le Tore Public devient la source directionnelle principale.

## **6.7. Synchronisation des IDs internes**

Les IDs internes sont créés lorsque :

- un chemin interne atteint une case spécifique,
- une séquence normal/fantôme cohérente est détectée,
- une progression exponentielle valide est observée.

### **6.7.1. Mise à jour des IDs**

Chaque clé :

- crée de nouveaux IDs,
- mute les IDs existants,
- met à jour l'ID global,
- renforce l'ID unique du tore.

### **6.7.2. Rôle des IDs dans la synchronisation**

Les IDs :

- garantissent la cohérence fractale,
- empêchent les dérivations,
- empêchent les falsifications,
- empêchent les répétitions,
- stabilisent la progression directionnelle.

## **6.8. Synchronisation de la Forteresse de Verre**

La Forteresse de Verre surveille :

- les déplacements orbitaux,
- les paires normal/fantôme,
- les chemins internes,
- les IDs internes,
- l'ID global,
- l'ID unique du tore,
- les ports directionnels,

- l'IP de confiance.

Elle valide la synchronisation à chaque clé.

## **6.9. Gestion des erreurs de synchronisation**

### **6.9.1. Incohérence fractale**

Si une incohérence est détectée :

- progression impossible,
- chemin illégitime,
- ID incohérent,
- mutation d'ID illégitime,
- inversion normal/fantôme illégitime,

la session est détruite.

### **6.9.2. Clé incohérente**

Si une clé :

- contient un ID invalide,
- contient un point de départ impossible,
- contient un point d'arrivée impossible,
- contient un type illégitime,

la session est détruite.

### **6.9.3. Absence de réponse**

Si aucune réponse après 5 secondes :

- une nouvelle clé est générée,
- avec un point de départ différent,
- mais reconstruisant le même fichier.

## **6.10. Résumé du Chapitre 6**

La synchronisation directionnelle :

- maintient la cohérence entre les trois tores,
- garantit la progression fractale,
- valide les IDs internes,
- met à jour l'ID global,
- stabilise l'ID unique du tore,
- assure la cohérence directionnelle,

- protège contre les dérivations, falsifications et répétitions,
- active la Forteresse de Verre,
- détruit la session en cas d'anomalie.

Elle constitue le mécanisme central qui permet au protocole de fonctionner comme une **architecture directionnelle inviolable**.

## Chapitre 7 — Multi-Tore et Réseaux Parallèles

### 7.1. Vue d'ensemble

Le multi-tore est l'un des fondements les plus puissants de l'architecture directionnelle. Il permet de créer **plusieurs réseaux directionnels indépendants**, chacun possédant :

- son propre Tore Privé,
- son propre Tore d'Authentification,
- ses propres clés directionnelles,
- ses propres IDs internes,
- son propre ID global,
- son propre ID unique du tore,
- sa propre Forteresse de Verre.

**Le Tore Public, lui, est pré-généré.** Il vient avec le logiciel du protocole et constitue la **base directionnelle universelle** pour toutes les branches.

Chaque réseau parallèle est **totalelement isolé** des autres, même s'ils coexistent :

- sur la même machine,
- sur le même système,
- dans le même environnement réseau.

Le multi-tore permet :

- la segmentation directionnelle,
- la souveraineté structurelle,
- l'isolation totale des branches,
- la création de réseaux privés inviolables,
- la duplication directionnelle contrôlée.

### 7.2. Définition d'un réseau parallèle

Un réseau parallèle est une **instance complète** de l'architecture directionnelle comprenant :

1. Le Tore Public (pré-généré, universel)

2. Un Tore d'Authentification
3. Un Tore Privé
4. Un ensemble de clés directionnelles
5. Une Forteresse de Verre
6. Un ID global de session
7. Un ID unique du tore

Chaque réseau parallèle est un **univers directionnel indépendant**.

Il ne partage :

- aucune clé,
- aucun ID,
- aucun tore privé,
- aucune progression fractale,
- aucune cohérence interne.

## 7.3. Création d'un nouveau tore

La création d'une nouvelle branche directionnelle suit trois étapes :

### 7.3.1. Utilisation du Tore Public pré-généré

Le Tore Public **n'est pas généré** lors de la création d'une branche. Il est :

- pré-généré,
- fourni avec le logiciel,
- universel,
- identique pour toutes les installations,
- non modifiable,
- non dérivable.

Il contient **toutes les combinaisons possibles des 30 bits**, ce qui en fait la base directionnelle universelle.

### 7.3.2. Génération du Tore d'Authentification

Un Tore d'Authentification de **1 000 000 cases × 3 bits** est généré. Il valide la première clé directionnelle et initialise :

- les premières paires normal/fantôme,
- les premiers chemins internes,
- l'ID global,
- l'ID unique du tore.

### 7.3.3. Génération du Tore Privé

Un Tore Privé de **1 000 000 cases × 3 bits** est généré. Il sera reconstruit fractalement à partir des clés directionnelles :

- inversion normal/fantôme,
- mutation d'ID,
- activation des chemins internes,
- progression orbitale.

Chaque Tore Privé est **indépendant** des autres tores privés existants.

## 7.4. Branches directionnelles

Chaque réseau parallèle est appelé **branche directionnelle**.

Une branche possède :

- son propre Tore Privé,
- sa propre progression fractale,
- ses propres IDs internes,
- sa propre Forteresse de Verre,
- son propre ID global,
- son propre ID unique du tore.

### 7.4.1. Branches indépendantes

Les branches ne communiquent pas entre elles. Elles ne peuvent pas :

- partager des clés,
- partager des IDs,
- partager des tores privés,
- partager des chemins internes,
- partager des mutations d'ID.

### 7.4.2. Branches souveraines

Chaque branche est souveraine :

- elle ne dépend d'aucune autre branche,
- elle ne peut pas être influencée par une autre branche,
- elle ne peut pas être dérivée par une autre branche.

## 7.5. Duplication directionnelle (Tore primaire hors-ligne)

Une fonctionnalité essentielle du multi-tore est la **duplication directionnelle hors-ligne**.

### 7.5.1. Tore primaire

Un tore primaire peut être :

- généré,
- stocké hors-ligne,
- transféré physiquement,
- utilisé pour créer une nouvelle branche.

### 7.5.2. Transfert hors-ligne

Le tore primaire peut être transféré :

- sur clé USB,
- sur disque dur,
- sur support physique,
- sur tout média non réseau.

Ce transfert ne contient **aucune clé directionnelle**. Il contient uniquement :

- la géométrie directionnelle,
- les blocs de 3 bits,
- les chemins internes.

### 7.5.3. Création d'une branche à partir du tore primaire

Une fois le tore primaire transféré :

- une nouvelle branche est créée,
- un nouveau Tore Privé est reconstruit,
- un nouvel ID global est généré,
- un nouvel ID unique du tore est établi,
- une nouvelle Forteresse de Verre est activée.

La branche est **totale**ment indépendante de la branche d'origine.

## 7.6. Isolation structurelle

L'isolation structurelle est totale.

### 7.6.1. Isolation fractale

Les actions fractales d'une branche :

- ne peuvent pas influencer une autre branche,
- ne peuvent pas dériver une autre branche,
- ne peuvent pas reconstruire une autre branche.

### 7.6.2. Isolation directionnelle

Les chemins internes d'une branche :

- ne peuvent pas être utilisés dans une autre branche,
- ne peuvent pas être dérivés par une autre branche.

### 7.6.3. Isolation des IDs

Les IDs internes :

- ne sont pas compatibles entre branches,
- ne peuvent pas être transférés,
- ne peuvent pas être dérivés.

## 7.7. Multi-tore et sécurité post-quantique

Le multi-tore renforce la sécurité post-quantique :

- un ordinateur quantique ne peut pas dériver un tore privé,
- ne peut pas dériver une branche,
- ne peut pas dériver une clé,
- ne peut pas dériver un ID,
- ne peut pas dériver l'ID unique du tore.

Même avec une puissance infinie, un attaquant ne peut pas :

- explorer un tore privé,
- dériver un tore privé,
- reconstruire un tore privé,
- analyser un tore privé.

Les tores privés sont **géométriquement inviolables**.

## 7.8. Multi-tore et sécurité photonique

Avec l'arrivée des ordinateurs photoniques :

- les structures mathématiques deviennent obsolètes,
- les cryptographies classiques deviennent vulnérables.

Le multi-tore est conçu pour cette ère :

- les tores privés sont non transmissibles,
- non dérivables,
- non analysables,
- non explorables.

Un ordinateur photonique ne peut pas :

- pénétrer un tore privé,
- analyser un tore privé,
- dériver un tore privé.

## 7.9. Résumé du Chapitre 7

Le multi-tore permet :

- la création de réseaux parallèles,
- l'isolation totale des branches,
- la duplication directionnelle hors-ligne,
- la souveraineté structurelle,
- la sécurité post-quantique,
- la sécurité photonique.

Chaque branche est un **univers directionnel indépendant**, inviolable par conception. Le Tore Public, pré-généré et universel, sert de base directionnelle commune, mais **jamais de tore privé partagé**.

# Chapitre 8 — Reconstruction de Fichiers Directionnels

## 8.1. Vue d'ensemble

La reconstruction de fichiers directionnels est l'une des fonctions majeures de l'architecture directionnelle. Elle permet de reconstruire un fichier **bit par bit**, en utilisant :

- les clés directionnelles (64 bits),
- les déplacements orbitaux dans le Tore Public,
- les mises à jour du Tore Privé,
- les blocs de **3 bits**,
- les IDs internes,
- l'ID global,
- l'ID unique du tore,
- la cohérence directionnelle,
- la Forteresse de Verre.

Contrairement aux systèmes classiques qui lisent un fichier séquentiellement, la reconstruction directionnelle est **structurelle, géométrique, non linéaire, non dérivable, et inviolable par conception**.

Chaque clé directionnelle reconstruit **exactement un bit** du fichier, selon une logique fractale et orbitale.

## 8.2. Types de clés impliquées dans la reconstruction

La reconstruction de fichiers utilise principalement deux types de clés :

| Type | Bits                      | Description                                                  |
|------|---------------------------|--------------------------------------------------------------|
| 00   | Reconstruction de fichier | Lit le Tore Public pour reconstruire un bit du fichier       |
| 01   | Clé suivie                | Une autre clé suit immédiatement pour maintenir la cohérence |

Les autres types interviennent dans des cas particuliers :

| Type | Bits                   | Description                                       |
|------|------------------------|---------------------------------------------------|
| 11   | Reconstruction de clés | Sert à reconstruire d'autres clés directionnelles |
| 10   | Clé d'arrêt            | Termine la reconstruction et stabilise le tore    |

## 8.3. Décomposition d'une clé de reconstruction (type 00)

Une clé de reconstruction contient :

- **8 bits** : point de départ ( $N^A E$ )
- **8 bits** : point d'arrivée ( $N^A E$ )
- **2 bits** : type de clé (00)
- **30 bits** : anneau orbital (segment de navigation)
- **16 bits** : ID de la case visée

Ces éléments définissent :

1. la fenêtre directionnelle,
2. la zone du Tore Public à parcourir,
3. la zone du Tore Privé à mettre à jour,
4. la mutation d'ID de la case de départ,
5. le bit du fichier à reconstruire.

## 8.4. Processus de reconstruction

La reconstruction suit un processus strict en **quatre étapes**.

### 8.4.1. Étape 1 — Détermination de la fenêtre directionnelle

Les 8 bits du point de départ et les 8 bits du point d'arrivée définissent une **fenêtre directionnelle** :

$$\text{Fenêtre} = [N_1^A E_1 \rightarrow N_2^A E_2]$$

Cette fenêtre détermine :

- la zone du Tore Public à parcourir,
- la zone du Tore Privé à mettre à jour,
- la zone du fichier à reconstruire.

#### 8.4.2. Étape 2 — Navigation orbitale dans le Tore Public

Les **30 bits orbitaux** de la clé sont transformés en :

- **60 lectures orbitale,**
- **180 blocs directionnels (N,E),**
- **180 déplacements exponentiels.**

Chaque déplacement lit **un bloc de 3 bits** dans le Tore Public :

1. lecture du bit normal/fantôme,
2. lecture du chemin interne (00, 01, 10, 11),
3. détermination de la prochaine case,
4. influence sur le Tore Privé.

La navigation orbitale est :

- non linéaire,
- non dérivable,
- non réversible,
- dépendante de l'état interne du tore.

#### 8.4.3. Étape 3 — Mise à jour du Tore Privé

Chaque bloc de **3 bits** lu dans le Tore Public :

- met à jour une case du Tore Privé,
- inverse normal/fantôme,
- active un chemin interne,
- crée ou valide un ID interne,
- **mute l'ID de la case de départ,**
- renforce l'ID unique du tore.

Le Tore Privé devient progressivement cohérent avec le fichier reconstruit.

#### 8.4.4. Étape 4 — Reconstruction du bit du fichier

Les mises à jour du Tore Privé permettent de reconstruire **un bit du fichier**.

Le bit reconstruit dépend :

- de l'ID de la case visée (16 bits),

- de la cohérence fractale,
- de la progression directionnelle,
- de l'état interne du Tore Privé,
- de la mutation d'ID,
- de l'inversion normal/fantôme.

Chaque clé reconstruit **exactement un bit**.

## 8.5. Reconstruction complète du fichier

Pour reconstruire un fichier de **N bits**, l'architecture utilise **N clés type 00**, éventuellement suivies de clés type 01.

### 8.5.1. Clés type 00

Chaque clé type 00 reconstruit **un bit**.

### 8.5.2. Clés type 01

Les clés type 01 indiquent :

**Une autre clé suit immédiatement.**

Elles permettent :

- d'enchaîner les clés,
- de maintenir la cohérence fractale,
- de garantir la continuité directionnelle.

### 8.5.3. Clé type 10 (arrêt)

Une clé type 10 :

- termine la reconstruction,
- valide le fichier reconstruit,
- met à jour l'ID global,
- stabilise le Tore Privé.

## 8.6. Protection contre les attaques

La reconstruction directionnelle est protégée par :

### 8.6.1. La Forteresse de Verre

Elle valide :

- les clés,
- les IDs,
- les chemins internes,

- les paires normal/fantôme,
- la cohérence fractale,
- la mutation d'ID.

### **8.6.2. Les tores**

Les tores sont :

- non dérivables,
- non transmissibles,
- non réversibles,
- non explorables.

### **8.6.3. Les clés directionnelles**

Elles sont :

- non prédictibles,
- non falsifiables,
- non injectables,
- non répétables.

### **8.6.4. La destruction directionnelle**

En cas d'anomalie :

- le Tore Privé est effacé,
- les IDs sont invalidés,
- les chemins internes sont détruits,
- l'ID global est effacé,
- la session est terminée.

## **8.7. Résumé du Chapitre 8**

La reconstruction de fichiers directionnels :

- utilise les clés type 00, 01 et 10,
- lit le Tore Public pré-généré,
- met à jour le Tore Privé,
- inverse normal/fantôme,
- mute les IDs internes,
- reconstruit un fichier bit par bit,
- repose sur la géométrie directionnelle,

- est protégée par la Forteresse de Verre,
- est inviolable par conception.

Elle constitue l'une des applications majeures de l'architecture directionnelle.

## Chapitre 9 — Reconstruction de Clés Directionnelles (Type 11)

*(Version corrigée et cohérente avec la mécanique directionnelle)*

### 9.1. Vue d'ensemble

La reconstruction de clés directionnelles est un processus interne permettant de **générer de nouvelles clés à partir de clés existantes**, selon une logique fractale, orbitale et directionnelle. Ce mécanisme est activé par les **clés de type 11**, qui ne reconstruisent **pas** un fichier, mais **reconstruisent d'autres clés**.

La reconstruction de clés est essentielle pour :

- la continuité directionnelle,
- la régénération en cas de perte de clé,
- la création de séquences complexes,
- la mise à jour des IDs internes,
- la cohérence fractale globale,
- la souveraineté structurelle du tore.

Elle permet au protocole de fonctionner **sans retransmission classique, sans dérivation mathématique, et sans cryptographie**.

### 9.2. Rôle des clés type 11

Les clés type 11 ont une fonction unique :

**Reconstruire une ou plusieurs clés directionnelles à partir de la géométrie fractale interne.**

Elles sont utilisées lorsque :

- une clé doit être régénérée,
- une clé doit être prolongée,
- une clé doit être remplacée,
- une clé doit être complétée,
- une clé doit être reconstruite pour maintenir la cohérence fractale.

Elles ne reconstruisent **jamais** un fichier. Elles reconstruisent **des clés**.

### 9.3. Structure d'une clé type 11

Une clé type 11 possède le même découpage que toutes les clés directionnelles :

- **8 bits** : point de départ ( $N^{\wedge}E$ )
- **8 bits** : point d'arrivée ( $N^{\wedge}E$ )
- **2 bits** : type de clé (11)
- **30 bits** : anneau orbital (segment de navigation)
- **16 bits** : ID de la case visée

#### ✓ Correction intégrée

- Les **30 bits** servent **toujours** à l'escalade des **10 niveaux orbitaux** dans le Tore Public.
- Le **point de départ** et le **point d'arrivée** servent **uniquement** à déterminer le point de départ et le point d'arrivée dans le Tore Privé.
- Ils n'ont **aucun rôle** dans le Tore Public.

### 9.4. Processus de reconstruction d'une clé

La reconstruction d'une clé directionnelle suit un processus strict en **quatre étapes**, identiques à celles des autres types de clés.

#### 9.4.1. Étape 1 — Détermination de la fenêtre directionnelle

Les 8 bits du point de départ et les 8 bits du point d'arrivée définissent une **fenêtre directionnelle** dans le Tore Privé :

$$\text{Fenêtre} = [N_1^{\wedge}E_1 \rightarrow N_2^{\wedge}E_2]$$

Cette fenêtre détermine :

- la zone du Tore Privé à parcourir,
- la zone du Tore Privé à mettre à jour,
- la zone directionnelle utilisée pour reconstruire la nouvelle clé.

#### ✓ Correction intégrée

La fenêtre **ne détermine jamais** la zone du Tore Public. Le Tore Public est toujours parcouru par les **30 bits orbitaux**, indépendamment du point de départ.

#### 9.4.2. Étape 2 — Navigation orbitale dans le Tore Public

Les **30 bits orbitaux** de la clé sont transformés en :

- **10 niveaux orbitaux**,
- **60 lectures orbitale**,
- **180 blocs directionnels (N,E)**,
- **180 déplacements exponentiels**.

Chaque déplacement lit **un bloc de 3 bits** dans le Tore Public :

1. lecture du bit normal/fantôme,
2. lecture du chemin interne (00, 01, 10, 11),
3. détermination de la prochaine case,
4. influence sur le Tore Privé.

La navigation orbitale est :

- non linéaire,
- non dérivable,
- non réversible,
- dépendante de l'état interne du tore.

### 9.4.3. Étape 3 — Lecture et mise à jour du Tore Privé

Une fois les 10 niveaux orbitaux exécutés, la clé lit les cases du Tore Privé selon la fenêtre directionnelle :

- mise à jour des paires normal/fantôme,
- activation des chemins internes,
- création ou validation d'IDs internes,
- **mutation de l'ID de la case de départ**,
- renforcement de l'ID unique du tore.

Ces mises à jour ne reconstruisent pas un fichier : elles reconstruisent **un état directionnel interne**.

### 9.4.4. Étape 4 — Génération de la nouvelle clé

Une fois les lectures du Tore Public et du Tore Privé terminées :

- l'état directionnel interne est stabilisé,
- les IDs internes sont mis à jour,
- l'ID global est ajusté,
- une **nouvelle clé directionnelle** est générée.

Cette nouvelle clé :

- possède un **nouveau point de départ**,
- un **nouveau point d'arrivée**,
- un **nouveau type** (00, 01, 10 ou 11 selon le contexte),
- une **nouvelle navigation orbitale**,
- un **nouvel ID de case visée**.

Elle est **structurellement cohérente** avec la clé précédente.

## 9.5. Cas particulier : régénération après perte de clé

Si une clé est perdue (aucune réponse après 5 secondes), le protocole ne retransmet **jamais** la même clé.

Il utilise une clé type 11 pour :

1. reconstruire une nouvelle clé,
2. avec un point de départ différent,
3. mais reconstruisant le **même fichier**,
4. ou poursuivant la **même séquence fractale**.

La clé type 11 garantit :

- la continuité fractale,
- la cohérence directionnelle,
- l'absence de collision,
- l'absence de dérivabilité.

## 9.6. Cas particulier : reconstruction de séquences complexes

Les clés type 11 sont utilisées pour :

- reconstruire des séquences de clés,
- prolonger des séquences,
- créer des séquences fractales complexes,
- stabiliser des séquences directionnelles.

Elles permettent de créer des structures directionnelles **impossibles à dériver ou à falsifier**.

## 9.7. Protection contre les attaques

La reconstruction de clés est protégée par :

### 9.7.1. La Forteresse de Verre

Elle valide :

- les clés type 11,
- les IDs internes,
- les chemins internes,
- les paires normal/fantôme,
- la cohérence fractale.

### 9.7.2. Les tores

Les tores sont :

- non dérivables,
- non transmissibles,
- non réversibles,
- non explorables.

### **9.7.3. Les clés directionnelles**

Elles sont :

- non prédictibles,
- non falsifiables,
- non injectables,
- non répétables.

### **9.7.4. La destruction directionnelle**

En cas d'anomalie :

- le Tore Privé est effacé,
- les IDs sont invalidés,
- les chemins internes sont détruits,
- l'ID global est effacé,
- la session est terminée.

## **9.8. Résumé du Chapitre 9**

La reconstruction de clés directionnelles :

- utilise les clés type 11,
- interprète les 30 bits orbitaux,
- escalade les 10 niveaux du Tore Public,
- lit ensuite le Tore Privé,
- inverse normal/fantôme,
- mute les IDs internes,
- génère de nouvelles clés,
- assure la continuité fractale,
- remplace les clés perdues,
- crée des séquences complexes,
- est protégée par la Forteresse de Verre,
- est inviolable par conception.

Elle constitue le mécanisme interne qui permet au protocole de fonctionner **sans retransmission classique** et **sans cryptographie**.

## Chapitre 10 — Clés d'Arrêt et Fin de Séquence Directionnelle

*(Version corrigée — aucune mention que la clé est une “clé de destruction”)*

### 10.1. Vue d'ensemble

Les clés d'arrêt (type **10**) constituent le mécanisme terminal d'une séquence directionnelle. Elles ne reconstruisent :

- ni fichier,
- ni clé,
- ni séquence fractale.

Elles ont une fonction unique :

**Terminer proprement une reconstruction directionnelle et signaler que la session doit être fermée.**

La fermeture de session déclenche ensuite un processus interne appelé **destruction directionnelle**, qui est un mécanisme structurel indépendant de la clé elle-même.

Ce chapitre décrit :

- la structure d'une clé d'arrêt,
- son rôle fractal,
- les conditions de fin de séquence,
- le processus de fermeture directionnelle,
- les protections structurelles associées.

### 10.2. Structure d'une clé d'arrêt (type 10)

Une clé type 10 possède le même découpage que toutes les clés directionnelles :

- **8 bits** : point de départ ( $N^E$ )
- **8 bits** : point d'arrivée ( $N^E$ )
- **2 bits** : type de clé (10)
- **30 bits** : anneau orbital (segment de navigation)
- **16 bits** : ID de la case visée

✓ Correction intégrée

Une clé type 10 suit **exactement les mêmes étapes fractales** que toutes les autres clés :

1. **Interprétation des 30 bits orbitaux**
2. **Escalade des 10 niveaux du Tore Public**
3. **Lecture des blocs dans le Tore Privé**
4. **Mutation d'ID**
5. **Inversion normal/fantôme**

La différence réside uniquement dans l'intention :

- elle ne reconstruit rien,
- elle ne génère pas de nouvelle clé,
- elle signale la fin de la séquence.

### **10.3. Rôle des clés d'arrêt**

Les clés d'arrêt ont **trois fonctions essentielles** :

#### **10.3.1. Terminer une reconstruction**

Une clé type 10 indique :

**La reconstruction est terminée.**

Elle valide :

- la fin d'un fichier,
- la fin d'une séquence de clés,
- la fin d'une branche directionnelle.

#### **10.3.2. Stabiliser l'ID global**

Avant fermeture, l'ID global est :

- finalisé,
- enregistré,
- stabilisé.

Il devient l'identifiant terminal de la session.

#### **10.3.3. Signaler la fermeture directionnelle**

Une clé type 10 indique au tore :

**La session doit être fermée.**

La fermeture directionnelle est un mécanisme interne indépendant, déclenché **après** la clé.

### **10.4. Déclenchement de la fermeture directionnelle**

La fermeture directionnelle est déclenchée lorsque :

- une clé type 10 est validée,

- les **5 transmissions** du handshake sont respectées,
- la Forteresse de Verre confirme la cohérence fractale,
- l’ID global est stabilisé.

Si une clé type 10 est incohérente :

- la session est fermée immédiatement,
- mais **sans stabilisation** de l’ID global.

## 10.5. Processus de fermeture directionnelle

La fermeture directionnelle se déroule en **quatre phases**.

*(Note : ce processus est interne. La clé type 10 ne “fait” pas ces actions — elle les déclenche.)*

### 10.5.1. Phase 1 — Incohérence volontaire des paires normal/fantôme

Les paires normal/fantôme du Tore Privé sont :

- inversées,
- mélangées,
- rendues incohérentes.

### 10.5.2. Phase 2 — Invalidation des chemins internes

Les chemins internes (00, 01, 10, 11) sont :

- désactivés,
- rendus incompatibles,
- rendus impossibles à suivre.

### 10.5.3. Phase 3 — Effacement des IDs internes

Les IDs internes :

- sont effacés,
- sont rendus incohérents,
- perdent leur relation fractale.

### 10.5.4. Phase 4 — Effacement du Tore Privé

Le Tore Privé est :

- effacé,
- désynchronisé,
- rendu structurellement illisible.

## 10.6. Protection contre les attaques

La fermeture directionnelle protège contre :

### 10.6.1. Analyse post-mortem

Après fermeture :

- aucun ID n'est cohérent,
- aucun chemin interne n'est valide,
- aucune paire normal/fantôme n'est exploitable,
- aucune progression fractale n'est reconstruisible.

### 10.6.2. Attaques quantiques

Un ordinateur quantique ne peut pas :

- explorer le tore,
- dériver le tore,
- reconstruire le tore.

### 10.6.3. Attaques photoniques

Un ordinateur photonique ne peut pas :

- analyser la géométrie interne,
- reconstruire les chemins internes,
- dériver les IDs.

## 10.7. Cas particulier : fermeture forcée

La fermeture directionnelle peut être déclenchée **sans clé type 10** si :

- une incohérence fractale est détectée,
- une clé est falsifiée,
- une clé est injectée,
- une clé est répétée,
- une clé est dérivée,
- une clé est incohérente.

Dans ce cas :

- l'ID global n'est pas stabilisé,
- la session est fermée immédiatement.

## 10.8. Résumé du Chapitre 10

Les clés d'arrêt :

- terminent une reconstruction,
- stabilisent l'ID global,

- signalent la fermeture directionnelle.

La fermeture directionnelle :

- rend incohérentes les paires normal/fantôme,
- invalide les chemins internes,
- efface les IDs internes,
- efface le Tore Privé,
- empêche toute analyse post-mortem,
- protège contre les attaques classiques, quantiques et photoniques.

Elle constitue le **mécanisme terminal** de l'architecture directionnelle, garantissant une inviolabilité structurelle totale.

# Chapitre 11 — Transmission Directionnelle Avancée (Host ↔ Client ↔ Host ↔ Client ↔ Host)

*(Version modernisée et cohérente avec les chapitres 1 à 10)*

## 11.1. Vue d'ensemble

La transmission directionnelle avancée est l'évolution complète du modèle Host ↔ Client ↔ Host introduit au Chapitre 5. Elle formalise :

- les règles avancées de transmission,
- les conditions de cohérence fractale,
- les mécanismes de correction directionnelle,
- les protections structurelles,
- les interactions avec les tores,
- les interactions avec la Forteresse de Verre,
- les cas particuliers (perte, duplication, multi-tore),
- les transmissions en cascade,
- les transmissions synchronisées.

Ce chapitre décrit **comment une clé directionnelle circule réellement dans un environnement fractal**, sans cryptographie, sans dérivation, sans retransmission classique.

## 11.2. Rappel de la séquence de base (version moderne)

La transmission directionnelle suit **toujours** la séquence complète à **5 transmissions** :

1. **Host** → **Client** : envoi de la clé
2. **Client** → **Host** : renvoi de la même clé
3. **Host** → **Client** : renvoi final de la même clé
4. **Client** → **Host** : confirmation d'exécution fractale
5. **Host** → **Client** : confirmation de synchronisation du tore

Cette séquence :

- valide la clé,
- valide l'IP du client,
- active la Forteresse de Verre,
- confirme l'exécution fractale,
- met à jour les inversions normal/fantôme,
- met à jour l'ID de la case de départ,
- synchronise les deux tores privés,
- poursuit la reconstruction fractale.

## 11.3. Transmission avancée : règles structurelles

### 11.3.1. Règle de cohérence fractale

Une clé est valide si :

- les **10 niveaux orbitaux** sont cohérents,
- les **180 blocs directionnels** sont compatibles,
- les paires normal/fantôme sont valides,
- les chemins internes sont cohérents,
- l'ID visé existe dans le Tore Privé,
- l'ID global reste stable.

### 11.3.2. Règle de continuité directionnelle

La clé doit :

- poursuivre la progression fractale,
- respecter les inversions fantômes,
- maintenir la cohérence des IDs internes,
- muter correctement l'ID de la case de départ.

### 11.3.3. Règle de non-répétition

Une clé ne peut **jamais** être répétée.

Si une clé identique est reçue deux fois :

- la session est détruite,
- le Tore Privé est effacé,
- l'IP est invalidée.

#### **11.3.4. Règle de non-dérivation**

Une clé ne peut pas être dérivée :

- par un attaquant,
- par un tore externe,
- par une autre branche.

### **11.4. Transmission avancée : interactions avec les tores**

#### **11.4.1. Interaction avec le Tore Public**

Le Tore Public :

- fournit les blocs de 3 bits,
- guide les déplacements orbitaux,
- influence la reconstruction du Tore Privé.

Chaque transmission lit **180 blocs de 3 bits** (issus des 30 bits orbitaux).

#### **11.4.2. Interaction avec le Tore Privé**

Le Tore Privé :

- valide les déplacements,
- met à jour les paires normal/fantôme,
- crée les IDs internes,
- mute l'ID de la case de départ,
- met à jour l'ID global.

Chaque transmission modifie **exactement une case** du Tore Privé.

#### **11.4.3. Interaction avec le Tore d'Authentification**

Le Tore d'Authentification :

- n'est utilisé qu'une seule fois,
- valide la première clé,
- initialise la cohérence directionnelle.

## **11.5. Transmission avancée : interactions avec la Forteresse de Verre**

La Forteresse de Verre surveille :

- les déplacements orbitaux,
- les paires normal/fantôme,
- les chemins internes,
- les IDs internes,
- l'ID global,
- les ports directionnels,
- l'IP de confiance.

Elle valide ou rejette chaque transmission.

### **11.5.1. Transmission valide**

Une transmission valide :

- poursuit la progression fractale,
- met à jour le Tore Privé,
- stabilise l'ID global,
- maintient la cohérence directionnelle.

### **11.5.2. Transmission invalide**

Une transmission invalide :

- détruit la session,
- efface le Tore Privé,
- invalide l'IP,
- désactive les chemins internes.

## **11.6. Transmission avancée : cas particuliers**

### **11.6.1. Perte de clé**

Si aucune réponse après 5 secondes :

- la clé est considérée perdue,
- une nouvelle clé est générée,
- avec un point de départ différent,
- mais reconstruisant le même fichier.

Aucune retransmission du même paquet n'est effectuée.

### **11.6.2. Transmission en cascade**

Certaines séquences nécessitent :

- une clé type 01 (une autre clé suit),
- suivie d'une clé type 00 ou 11.

La transmission en cascade :

- maintient la cohérence fractale,
- permet des reconstructions complexes,
- stabilise les IDs internes.

### **11.6.3. Transmission multi-tore**

Dans un environnement multi-tore :

- chaque branche possède sa propre transmission,
- les transmissions ne se croisent jamais,
- les clés ne sont pas compatibles entre branches.

Une clé d'une branche ne peut pas être utilisée dans une autre branche.

### **11.6.4. Transmission synchronisée**

Certaines séquences nécessitent une synchronisation :

- entre plusieurs clés,
- entre plusieurs IDs,
- entre plusieurs chemins internes.

La transmission synchronisée :

- stabilise l'ID global,
- crée des structures fractales complexes,
- permet des reconstructions directionnelles avancées.

## **11.7. Transmission avancée : protections structurelles**

La transmission avancée protège contre :

### **11.7.1. Attaques classiques**

Impossible de :

- falsifier une clé,
- injecter une clé,
- répéter une clé,
- dériver une clé.

### 11.7.2. Attaques quantiques

Impossible de :

- explorer le tore,
- dériver le tore,
- reconstruire le tore.

### 11.7.3. Attaques photoniques

Impossible de :

- analyser la géométrie interne,
- reconstruire les chemins internes,
- dériver les IDs.

## 11.8. Résumé du Chapitre 11

La transmission directionnelle avancée :

- étend le modèle Host ↔ Client ↔ Host ↔ Client ↔ Host,
- ajoute des règles fractales,
- maintient la cohérence directionnelle,
- protège contre les dérivations, falsifications et répétitions,
- interagit avec les trois tores,
- interagit avec la Forteresse de Verre,
- gère les pertes, cascades et synchronisations,
- fonctionne dans un environnement multi-tore,
- est inviolable par conception.

Elle constitue le **mécanisme de communication fondamental** de l'architecture directionnelle.

# Chapitre 12 — Ports Directionnels et Géométrie Interne de Communication

## 12.1. Vue d'ensemble

Les ports directionnels constituent l'un des aspects les plus uniques et révolutionnaires de l'architecture directionnelle. Contrairement aux ports classiques (TCP/UDP), qui sont des entiers compris entre 0 et 65535, les ports directionnels sont des **structures géométriques internes**, définies par :

- les chemins internes (00, 01, 10, 11),
- les paires normal/fantôme,

- les IDs internes,
- l'ID global,
- la position fractale dans le Tore Privé,
- la fenêtre directionnelle ( $N^A E \rightarrow N^A E$ ),
- la cohérence directionnelle validée par la Forteresse de Verre.

Un port directionnel n'est :

- ni un nombre,
- ni un identifiant réseau,
- ni un canal TCP,
- ni un socket,
- ni scannable.

Un port directionnel est une **géométrie interne vivante**, impossible à dériver, impossible à explorer, impossible à falsifier.

## 12.2. Définition d'un port directionnel

Un port directionnel est une structure fractale composée de :

1. **Un chemin interne** (00, 01, 10, 11)
2. **Une paire normal/fantôme**
3. **Un ID interne**
4. **Un ID global**
5. **Une position fractale dans le Tore Privé**
6. **Une fenêtre directionnelle** ( $N^A E \rightarrow N^A E$ )
7. **Une cohérence fractale validée par la Forteresse de Verre**

Un port directionnel est donc un **objet géométrique**, non un objet numérique.

## 12.3. Pourquoi les ports directionnels sont inviolables

### 12.3.1. Ils n'existent pas dans la couche réseau classique

Un attaquant ne peut pas :

- scanner les ports,
- sonder les ports,
- envoyer des paquets vers un port,
- ouvrir un port,
- fermer un port.

Les ports directionnels n'existent que dans la **géométrie interne du tore**.

### **12.3.2. Ils sont définis par la mécanique fractale**

Un port directionnel dépend :

- des **10 niveaux orbitaux**,
- des **180 blocs directionnels**,
- des **blocs de 3 bits**,
- des chemins internes,
- des paires normal/fantôme,
- des IDs internes.

Impossible à dériver.

### **12.3.3. Ils sont validés par la Forteresse de Verre**

La Forteresse de Verre :

- surveille les ports directionnels,
- valide leur cohérence,
- détruit la session si un port est violé.

### **12.3.4. Ils sont non transmissibles**

Un port directionnel :

- ne peut pas être envoyé,
- ne peut pas être copié,
- ne peut pas être exporté,
- ne peut pas être dérivé.

Il n'existe que **localement**, dans le tore.

## **12.4. Structure interne d'un port directionnel**

Un port directionnel est composé de **sept couches fractales**.

### **12.4.1. Couche 1 — Chemin interne**

Le chemin interne (00, 01, 10, 11) détermine :

- la direction du port,
- la géométrie du port,
- la progression fractale du port.

### **12.4.2. Couche 2 — Paire normal/fantôme**

La paire normal/fantôme détermine :

- la nature du port,
- la validité du port,
- la cohérence du port.

Un port fantôme est inversé. Un port normal est direct.

### 12.4.3. Couche 3 — ID interne

Chaque port possède un **ID interne**, créé par :

- les déplacements orbitaux,
- les blocs de 3 bits,
- les chemins internes.

Cet ID est unique, non dérivable, non transmissible.

### 12.4.4. Couche 4 — ID global

L'ID global :

- relie le port à la session,
- stabilise la géométrie interne,
- empêche les collisions.

### 12.4.5. Couche 5 — Position dans le Tore Privé

Chaque port est associé à une case du Tore Privé :

- 1 000 000 cases,
- structure directionnelle interne.

La position du port est fractale, non linéaire.

### 12.4.6. Couche 6 — Fenêtre directionnelle

La fenêtre directionnelle ( $N^{\wedge}E \rightarrow N^{\wedge}E$ ) détermine :

- la zone du Tore Privé utilisée,
- la zone fractale du port,
- la mutation d'ID de la case de départ.

*(Note : la fenêtre ne détermine jamais la zone du Tore Public.)*

### 12.4.7. Couche 7 — Cohérence fractale

La cohérence fractale est validée par :

- les 10 niveaux orbitaux,
- les paires normal/fantôme,
- les chemins internes,

- les IDs internes,
- l'ID global.

Un port incohérent détruit la session.

## 12.5. Ports directionnels et transmission

Les ports directionnels interviennent dans la transmission :

### 12.5.1. Host → Client

Le host ouvre un port directionnel interne. Le client doit répondre sur **le même port directionnel**.

### 12.5.2. Client → Host

Le client renvoie la clé sur ce port. Le host valide la cohérence fractale.

### 12.5.3. Host → Client

Le host renvoie la clé finale sur le même port.

Si un port directionnel est violé :

- la session est détruite,
- le Tore Privé est effacé,
- l'IP est invalidée.

## 12.6. Ports directionnels et multi-tore

Dans un environnement multi-tore :

- chaque branche possède ses propres ports directionnels,
- les ports ne sont pas compatibles entre branches,
- les ports ne peuvent pas être dérivés d'une branche à l'autre.

Un port directionnel est **strictement local** à sa branche.

## 12.7. Ports directionnels et sécurité post-quantique

Les ports directionnels sont inviolables par un ordinateur quantique :

- impossible de dériver la géométrie interne,
- impossible d'explorer le tore,
- impossible de reconstruire les chemins internes,
- impossible de dériver les IDs.

## 12.8. Ports directionnels et sécurité photonique

Les ordinateurs photoniques peuvent :

- analyser des structures mathématiques,

- dériver des clés cryptographiques,
- casser des protocoles classiques.

Mais ils ne peuvent pas :

- analyser une géométrie directionnelle,
- dériver un tore,
- reconstruire un port directionnel.

Les ports directionnels sont **structurellement inviolables**.

## 12.9. Résumé du Chapitre 12

Les ports directionnels :

- ne sont pas des ports TCP,
- sont des structures fractales internes,
- sont définis par les chemins internes,
- sont définis par les paires normal/fantôme,
- sont définis par les IDs internes,
- sont validés par la Forteresse de Verre,
- sont inviolables par conception,
- ne peuvent pas être scannés,
- ne peuvent pas être dérivés,
- ne peuvent pas être falsifiés.

Ils constituent la **couche de communication interne** de l'architecture directionnelle, totalement indépendante des protocoles réseau classiques.

# Chapitre 13 — Géométrie Interne du Tore Privé (Structure Vivante)

## 13.1. Vue d'ensemble

Le Tore Privé est **l'organe vivant** de TorusNet. Contrairement :

- au Tore Public (statique, universel, pré-généré),
- au Tore d'Authentification (statique, utilisé une seule fois),

le Tore Privé est :

- vivant,
- évolutif,

- directionnel,
- fractalo-géométrie,
- non dérivable,
- non transmissible,
- non réversible,
- non analysable,
- non exploré,
- non explorables même par un ordinateur quantique ou photonique.

Il constitue la **structure interne souveraine** de chaque session TorusNet.

## 13.2. Structure de base du Tore Privé

Le Tore Privé est un tableau de :

**1 000 000 cases (1000 × 1000)**

Chaque case contient :

- **un état normal ou fantôme,**
- **4 chemins internes prédéfinis,**
- **un ID de case (16 bits),**
- **une relation avec l'ID global.**

### ✓ Règle fondamentale

Les **3 bits** issus de l'escalade des 10 niveaux orbitaux sont interprétés ainsi :

- **Bit 0** → indique si le **bit fantôme** doit être lu
- **Bits 1–2** → sélectionnent **l'un des 4 chemins internes** de la case

Ainsi, chaque case est un **nœud directionnel à 4 sorties**, contrôlé par la géométrie fractale.

## 13.3. Le Tore Privé comme structure vivante

### 13.3.1. Il évolue à chaque clé directionnelle

Chaque clé :

- sélectionne un chemin interne via les 3 bits orbitaux,
- modifie une case,
- modifie l'état normal/fantôme,
- active un des 4 chemins internes,
- crée ou valide un ID interne,
- mute l'ID de la case de départ,

- met à jour l'ID global.

### **13.3.2. Il possède une mémoire directionnelle**

Cette mémoire est géométrique :

- dans les états normal/fantôme,
- dans les 4 chemins internes,
- dans les IDs internes,
- dans l'ID global.

### **13.3.3. Il possède une cohérence interne**

La cohérence interne est :

- fractale,
- directionnelle,
- non linéaire,
- non mathématique.

### **13.3.4. Il possède une dynamique interne**

Le Tore Privé :

- se déplace,
- se réorganise,
- se rééquilibre,
- se stabilise,
- se détruit.

## **13.4. Les trois couches internes du Tore Privé**

### **13.4.1. Couche 1 — Géométrie normal/fantôme**

Définit :

- la nature des déplacements,
- la validité des chemins,
- la cohérence fractale.

Un état fantôme est une inversion directionnelle.

### **13.4.2. Couche 2 — Chemins internes (4 par case)**

Chaque case contient **4 chemins internes** :

- un chemin local,
- un chemin diagonal,

- un chemin exponentiel,
- un chemin directionnel complexe.

Les **3 bits orbitaux** sélectionnent le chemin :

- **Bit 0** → lecture fantôme activée ou non
- **Bits 1–2** → direction interne (00, 01, 10, 11)

Le chemin **10** (exponentiel) est le plus puissant : → il crée des IDs internes.

### 13.4.3. Couche 3 — IDs internes et ID global

Les IDs internes sont créés lorsque :

- un chemin interne atteint une case spécifique,
- une séquence normal/fantôme cohérente est détectée,
- une progression exponentielle valide est observée.

#### Partition directionnelle des 16 bits

Les 16 bits de l'ID de case sont divisés en deux groupes :

1. **32 768 valeurs** → cases de départ valides
2. **32 768 valeurs** → permutations directionnelles d'ID

#### Définition du nouvel ID de case

Lorsqu'une clé est exécutée :

- les **premiers éléments reconstruits dans le Tore Privé** définissent le **nouvel ID de la case**.

Ce nouvel ID :

- remplace l'ancien,
- est choisi parmi les 32 768 valeurs de permutation,
- dépend de la géométrie fractale produite par la clé,
- est impossible à prédire ou dériver.

#### ID global

L'ID global :

- résume l'état du tore,
- stabilise la session,
- garantit la cohérence directionnelle.

## 13.5. Géométrie interne : dynamique fractale

### 13.5.1. Règle de cohérence

Un déplacement est valide si :

- l'état normal/fantôme est cohérent,
- le chemin interne sélectionné est compatible,
- l'ID interne est valide,
- l'ID global reste stable.

### 13.5.2. Règle de continuité

La progression doit :

- rester dans les limites du tore,
- suivre les chemins internes,
- respecter les inversions fantômes.

### 13.5.3. Règle d'expansion

Les déplacements exponentiels :

- accélèrent la progression,
- créent des IDs internes,
- augmentent la complexité fractale.

## 13.6. Interaction avec les clés directionnelles

Chaque clé directionnelle :

- interprète les **30 bits orbitaux**,
- escalade les **10 niveaux du Tore Public**,
- produit une **séquence directionnelle** découpée en blocs de 3 bits,
- **Bit 0** → indique si le bit fantôme doit être lu,
- **Bits 1–2** → sélectionnent l'un des 4 chemins internes de la case,
- met à jour une case du Tore Privé,
- utilise les 16 bits pour déterminer la case de départ,
- utilise les premiers éléments reconstruits pour définir le nouvel ID de la case,
- remplace l'ancien ID,
- modifie la géométrie interne,
- crée ou valide des IDs internes,
- met à jour l'ID global.

Le Tore Privé est entièrement reconstruit par les clés.

## 13.7. Interaction avec la Forteresse de Verre

La Forteresse de Verre surveille :

- les états normal/fantôme,
- les chemins internes,
- les IDs internes,
- l'ID global,
- les ports directionnels.

Si une incohérence est détectée :

- la session est détruite,
- le Tore Privé est effacé.

## **13.8. Interaction avec les ports directionnels**

Les ports directionnels sont des structures internes du Tore Privé.

Ils dépendent :

- des 4 chemins internes,
- des états normal/fantôme,
- des IDs internes,
- de l'ID global.

Un port directionnel est une géométrie interne, non un nombre.

## **13.9. Interaction avec le multi-tore**

Chaque branche possède son propre Tore Privé.

Les tores privés :

- ne sont pas compatibles entre branches,
- ne peuvent pas être dérivés,
- ne peuvent pas être transférés,
- ne peuvent pas être analysés.

Chaque Tore Privé est un univers directionnel indépendant.

## **13.10. Destruction directionnelle du Tore Privé**

La destruction directionnelle :

- rend incohérentes les paires normal/fantôme,
- invalide les chemins internes,
- efface les IDs internes,
- efface l'ID global,
- efface le Tore Privé.

Elle empêche toute analyse post-mortem.

## 13.11. Résumé du Chapitre 13

Le Tore Privé :

- est une structure vivante,
- évolue à chaque clé,
- possède **4 chemins internes par case**,
- utilise les **3 bits orbitaux** pour sélectionner le chemin,
- possède des états normal/fantôme,
- possède des IDs internes,
- possède un ID global,
- mute l'ID de la case de départ à chaque clé,
- définit le nouvel ID via les premiers éléments reconstruits,
- interagit avec les clés,
- interagit avec la Forteresse de Verre,
- interagit avec les ports directionnels,
- est inviolable par conception,
- est détruit en cas d'anomalie.

Il constitue le **cœur vivant** de TorusNet.

## Chapitre 14 — Géométrie Interne du Tore Public (Structure Directionnelle Statique)

(Version : le Tore Public contient *toutes les combinaisons possibles sur 30 bits*)

### 14.1. Vue d'ensemble

Le Tore Public est la **structure directionnelle statique et exhaustive** de TorusNet. Contrairement :

- au Tore Privé (vivant, évolutif),
- au Tore d'Authentification (utilisé une seule fois),

le Tore Public est :

- immuable,
- universel,
- complet,
- non modifiable,

- non réversible,
- non dérivable,
- non transmissible,
- non analysable,
- non exploré,
- non explorables même par un ordinateur quantique ou photonique.

Il constitue la **source directionnelle totale** de toutes les clés, de toutes les progressions fractales, de toutes les reconstructions, et de toutes les validations structurelles.

## 14.2. Structure de base du Tore Public

Le Tore Public contient :

**toutes les combinaisons possibles des 30 bits** soit :  $2^{30} = 1\,073\,741\,824$  **cases directionnelles uniques**

Chaque case contient **exactement 30 bits**, découpés en :

- **10 blocs de 3 bits**,
- **aucun bit inutilisé.**

### Interprétation des 3 bits

Chaque bloc de 3 bits suit la règle directionnelle :

- **Bit 0** : normal (0) ou fantôme (1)
- **Bits 1–2** : direction interne (00, 01, 10, 11)

Ces blocs définissent :

- les sélecteurs fractals,
- les inversions normal/fantôme,
- les directions internes,
- les signaux de progression,
- les mutations d'ID dans le Tore Privé,
- la cohérence directionnelle globale.

## 14.3. Nature statique du Tore Public

Le Tore Public est **totalement statique**, c'est-à-dire :

### 14.3.1. Il ne change jamais

Aucune clé directionnelle ne peut :

- modifier le Tore Public,

- altérer le Tore Public,
- réécrire le Tore Public.

### **14.3.2. Il ne dépend d'aucune session**

Le Tore Public :

- est identique pour toutes les sessions,
- est identique pour tous les clients,
- est identique pour tous les hosts.

### **14.3.3. Il ne dépend d'aucune branche**

Dans un environnement multi-tore :

- chaque branche utilise le même Tore Public,
- le Tore Public est statique.

### **14.3.4. Il ne dépend d'aucune clé**

Les clés directionnelles :

- lisent le Tore Public,
- mais ne le modifient jamais.

## **14.4. Géométrie interne du Tore Public**

La géométrie interne du Tore Public est définie par :

1. les **10 blocs de 3 bits**,
2. les inversions normal/fantôme,
3. les directions internes (00, 01, 10, 11),
4. les séquences fractales,
5. les fenêtres directionnelles ( $N^{\wedge}E \rightarrow N^{\wedge}E$ ),
6. les signaux orbitaux produits par les clés.

Cette géométrie est :

- non linéaire,
- non mathématique,
- non dérivable,
- non réversible,
- non explorée,
- non explorables même par un ordinateur quantique ou photonique.

## 14.5. Rôle du Tore Public dans la mécanique fractale

Le Tore Public est la **source directionnelle totale** de la mécanique fractale.

Chaque clé directionnelle :

- interprète les **30 bits de navigation**,
- escalade les **10 niveaux orbitaux**,
- produit **10 blocs de 3 bits**,
- lit normal/fantôme via le bit 0,
- lit la direction interne via les bits 1–2,
- transmet ces signaux au Tore Privé,
- déclenche les mises à jour internes,
- crée ou valide les IDs internes,
- met à jour l'ID global.

Le Tore Public est donc la **base directionnelle statique** de TorusNet.

## 14.6. Interaction entre Tore Public et Tore Privé

La relation entre les deux tores est **asymétrique** :

### 14.6.1. Le Tore Public influence le Tore Privé

Le Tore Public :

- guide les déplacements fractals,
- influence les mises à jour du Tore Privé,
- influence la création des IDs internes,
- influence la mutation des IDs de case,
- influence la mise à jour de l'ID global.

### 14.6.2. Le Tore Privé n'influence jamais le Tore Public

Le Tore Privé :

- ne peut pas modifier le Tore Public,
- ne peut pas dériver le Tore Public,
- ne peut pas reconstruire le Tore Public.

## 14.7. Interaction entre Tore Public et clés directionnelles

Les clés directionnelles utilisent le Tore Public pour :

- reconstruire des fichiers,
- reconstruire des clés,

- reconstruire des séquences fractales,
- stabiliser l’ID global,
- valider la cohérence directionnelle.

Les **30 bits de navigation** d’une clé :

- sont découpés en **10 blocs de 3 bits**,
- chaque bloc sélectionne un chemin interne dans le Tore Privé,
- chaque bloc influence la progression fractale.

## 14.8. Interaction entre Tore Public et Forteresse de Verre

La Forteresse de Verre utilise le Tore Public pour :

- valider les signaux orbitaux,
- valider les directions internes,
- valider les inversions normal/fantôme,
- valider les IDs internes,
- valider l’ID global.

Si une incohérence est détectée :

- la session est détruite,
- le Tore Privé est effacé.

## 14.9. Interaction entre Tore Public et multi-tore

Dans un environnement multi-tore :

- chaque branche utilise le même Tore Public,
- le Tore Public est statique,
- le Tore Public est indépendant,
- le Tore Public est inviolable.

## 14.10. Résumé du Chapitre 14

Le Tore Public :

- est statique,
- est universel,
- est immuable,
- contient **toutes les combinaisons possibles des 30 bits**,
- soit **1 073 741 824 cases directionnelles**,
- contient 10 blocs de 3 bits par case,

- définit la géométrie directionnelle,
- guide les déplacements fractals,
- influence le Tore Privé,
- est validé par la Forteresse de Verre,
- est non dérivable,
- est non transmissible,
- est non réversible,
- est non analysable,
- est non exploré,
- est non explorables même par un ordinateur quantique ou photonique.

Il constitue la **base directionnelle statique totale** de TorusNet.

## Chapitre 15 — Géométrie Interne du Tore d'Authentification (Structure Directionnelle Initiale)

*(Version : même structure de case que le Tore Privé)*

### 15.1. Vue d'ensemble

Le Tore d'Authentification est la **structure directionnelle initiale** de TorusNet. Contrairement :

- au Tore Public (statique, universel, exhaustif),
- au Tore Privé (vivant, évolutif),

le Tore d'Authentification est :

- statique,
- local,
- unique à chaque branche,
- utilisé une seule fois,
- non modifiable,
- non dérivable,
- non transmissible,
- non réversible,
- non analysable,
- non exploré,

- non explorables même par un ordinateur quantique ou photonique.

Il constitue la **porte d'entrée directionnelle** de TorusNet. C'est lui qui valide la **première clé directionnelle** et initialise la **cohérence fractale**.

## 15.2. Structure de base du Tore d'Authentification

Le Tore d'Authentification possède **exactement la même structure que le Tore Privé** :

**1 000 000 cases (1000 × 1000) chaque case contient :**

- un état normal/fantôme,
- **4 chemins internes prédéfinis**,
- un ID de case (16 bits),
- une relation avec l'ID global,
- **10 blocs de 3 bits** servant de sélecteurs directionnels.

### Interprétation des 3 bits

Chaque bloc de 3 bits suit la règle directionnelle :

- **Bit 0** : indique si le bit fantôme doit être lu
- **Bits 1–2** : sélectionnent l'un des **4 chemins internes** de la case

Ces blocs définissent :

- la géométrie directionnelle initiale,
- les premiers déplacements fractals,
- les premières inversions normal/fantôme,
- les premiers chemins internes,
- les premiers IDs internes,
- l'ID global initial.

## 15.3. Rôle du Tore d'Authentification

Le Tore d'Authentification possède **trois rôles essentiels** :

### 15.3.1. Valider la première clé directionnelle

La première clé directionnelle est validée par :

- les blocs de 3 bits du Tore d'Authentification,
- les chemins internes sélectionnés,
- les inversions normal/fantôme,
- les premiers IDs internes,
- la cohérence fractale initiale.

### 15.3.2. Initialiser le Tore Privé

Le Tore d'Authentification :

- initialise les états normal/fantôme du Tore Privé,
- initialise les 4 chemins internes de chaque case,
- initialise les IDs internes,
- initialise l'ID global.

### 15.3.3. Activer la Forteresse de Verre

La Forteresse de Verre est activée lorsque :

- la première clé est validée,
- la cohérence fractale initiale est confirmée,
- l'IP du client devient une **IP de confiance**.

## 15.4. Géométrie interne du Tore d'Authentification

La géométrie interne du Tore d'Authentification est définie par :

1. les **10 blocs de 3 bits**,
2. les directions internes (00, 01, 10, 11),
3. les inversions normal/fantôme,
4. les séquences fractales initiales,
5. les fenêtres directionnelles ( $N^E \rightarrow N^E$ ),
6. les signaux orbitaux de la première clé.

Cette géométrie est :

- non linéaire,
- non mathématique,
- non dérivable,
- non réversible,
- non explorée,
- non explorables même par un ordinateur quantique ou photonique.

## 15.5. Interaction entre Tore d'Authentification et première clé

La première clé directionnelle :

- interprète les **30 bits orbitaux**,
- escalade les **10 niveaux directionnels**,
- produit **10 blocs de 3 bits**,

- lit normal/fantôme via le bit 0,
- lit la direction interne via les bits 1–2,
- initialise le Tore Privé,
- crée les premiers IDs internes,
- crée l’ID global initial.

Le Tore d’Authentification est donc la **base directionnelle initiale** de TorusNet.

## 15.6. Interaction entre Tore d’Authentification et Tore Privé

La relation entre les deux tores est **asymétrique** :

### 15.6.1. Le Tore d’Authentification influence le Tore Privé

Le Tore d’Authentification :

- initialise les états normal/fantôme,
- initialise les 4 chemins internes,
- initialise les IDs internes,
- initialise l’ID global.

### 15.6.2. Le Tore Privé n’influence jamais le Tore d’Authentification

Le Tore Privé :

- ne peut pas modifier le Tore d’Authentification,
- ne peut pas dériver le Tore d’Authentification,
- ne peut pas reconstruire le Tore d’Authentification.

## 15.7. Interaction entre Tore d’Authentification et Forteresse de Verre

La Forteresse de Verre utilise le Tore d’Authentification pour :

- valider la première clé,
- valider les premiers déplacements fractals,
- valider les inversions normal/fantôme,
- valider les premiers chemins internes,
- valider les premiers IDs internes,
- valider l’ID global initial.

Si une incohérence est détectée :

- la session est détruite,
- le Tore Privé est effacé.

## 15.8. Interaction entre Tore d'Authentification et multi-tore

Dans un environnement multi-tore :

- chaque branche possède son propre Tore d'Authentification,
- chaque Tore d'Authentification est statique,
- chaque Tore d'Authentification est indépendant,
- chaque Tore d'Authentification est inviolable.

## 15.9. Résumé du Chapitre 15

Le Tore d'Authentification :

- est statique,
- est local,
- est unique à chaque branche,
- possède **exactement la même structure de case que le Tore Privé**,
- possède **1 000 000 cases**,
- chaque case contient :
  - un état normal/fantôme,
  - **4 chemins internes**,
  - un ID de case,
  - une relation avec l'ID global,
  - **10 blocs de 3 bits**,
- définit la géométrie directionnelle initiale,
- valide la première clé,
- initialise le Tore Privé,
- crée les premiers IDs internes,
- crée l'ID global,
- active la Forteresse de Verre,
- est non dérivable,
- est non transmissible,
- est non réversible,
- est non analysable,
- est non exploré,
- est non explorables même par un ordinateur quantique ou photonique.

Il constitue la **structure directionnelle initiale** de TorusNet.

# Chapitre 16 — Cohérence Directionnelle Globale (ID Global et Stabilité Fractale)

## 16.1. Vue d'ensemble

La cohérence directionnelle globale est le mécanisme qui garantit que l'ensemble de la session TorusNet reste :

- stable,
- cohérent,
- inviolable,
- non dérivable,
- non réversible,
- non falsifiable,
- non analysable,
- non explorables même par un ordinateur quantique ou photonique.

Ce mécanisme repose sur un élément central :

**L'ID global**, une structure directionnelle vivante qui résume l'état fractal complet du Tore Privé.

L'ID global n'est :

- ni un nombre,
- ni une clé,
- ni un hash,
- ni une signature,
- ni une structure mathématique.

C'est une **structure directionnelle émergente**, dérivée de :

- la géométrie interne du Tore Privé,
- les paires normal/fantôme,
- les 4 chemins internes,
- les IDs internes,
- les fenêtres directionnelles,
- les déplacements fractals,
- les clés directionnelles validées,

- la Forteresse de Verre.

## 16.2. Définition de l’ID global

L’ID global est une **structure fractale vivante**, mais son **encodage directionnel** est parfaitement défini :

**L’ID global est un ID directionnel de 16 bits, construit à partir du second segment du parcours fractal dans le Tore Privé. Le premier segment de 16 bits reconstruit définit l’ID de la case de départ. Le second segment de 16 bits définit l’ID global. Les deux segments sont successifs, indépendants, et produits par le même parcours.**

### Origine des deux segments

- **Bits 0–15 → ID de la case de départ** (première fenêtre du parcours fractal)
- **Bits 16–31 → ID global** (seconde fenêtre du parcours fractal)

### Espace de valeurs

- Les IDs de case utilisent leurs propres plages de 16 bits.
- **L’ID global utilise une valeur libre parmi les 16 bits non utilisées par les IDs de case.**

L’ID global est donc :

- un ID directionnel 16 bits,
- vivant,
- mis à jour à chaque clé,
- impossible à dériver ou rejouer.

## 16.3. Rôle de l’ID global

### 16.3.1. Cohérence fractale de la session

Il garantit que :

- les clés sont cohérentes,
- les déplacements fractals sont cohérents,
- les IDs internes sont cohérents,
- les chemins internes sont cohérents,
- les paires normal/fantôme sont cohérentes.

### 16.3.2. Stabilité directionnelle

Il stabilise :

- la progression fractale,
- les mises à jour du Tore Privé,

- les ports directionnels,
- les séquences de clés.

### 16.3.3. Protection structurelle

Il empêche :

- les dérivations,
- les falsifications,
- les répétitions,
- les injections,
- les reconstructions illégitimes.

### 16.3.4. Destruction directionnelle

En cas d'anomalie :

- l'ID global est invalidé,
- la session est détruite,
- le Tore Privé est effacé.

## 16.4. Création de l'ID global

L'ID global est créé lors de la validation de la **première clé directionnelle**, grâce au Tore d'Authentification.

La création repose sur :

1. le premier segment de 16 bits (ID de case),
2. le second segment de 16 bits (ID global),
3. les paires normal/fantôme initiales,
4. les chemins internes initiaux,
5. les premiers IDs internes,
6. la première fenêtre directionnelle  $N^{\wedge}E \rightarrow N^{\wedge}E$ ,
7. la validation par la Forteresse de Verre.

L'ID global est une **émergence fractale**, non un calcul.

## 16.5. Mise à jour de l'ID global

Chaque clé directionnelle met à jour l'ID global en deux étapes successives :

### 16.5.1. Reconstruction du premier ID (ID de case)

Le parcours fractal reconstruit un ID de 16 bits :

**ID\_case = bits 0–15**

### 16.5.2. Reconstruction du second ID (ID global)

Le parcours continue immédiatement et reconstruit un second ID de 16 bits :

**ID\_global = bits 16–31**

### 16.5.3. Validation directionnelle

La Forteresse de Verre :

- valide les deux IDs,
- valide la cohérence fractale,
- valide la cohérence directionnelle,
- stabilise l'ID global.

## 16.6. Stabilité fractale

La stabilité fractale est assurée par :

- la cohérence des paires normal/fantôme,
- la cohérence des chemins internes,
- la cohérence des IDs internes,
- la cohérence des fenêtres directionnelles,
- la cohérence des déplacements fractals.

## 16.7. Interaction entre ID global et Forteresse de Verre

La Forteresse de Verre :

- surveille l'ID global,
- valide l'ID global,
- stabilise l'ID global,
- détruit la session si l'ID global est incohérent.

L'ID global est la **clé de voûte de la sécurité directionnelle**.

## 16.8. Interaction entre ID global et multi-tore

Dans un environnement multi-tore :

- chaque branche possède son propre ID global,
- les IDs globaux ne sont pas compatibles entre branches,
- les IDs globaux ne peuvent pas être dérivés,
- les IDs globaux ne peuvent pas être transférés.

Chaque ID global est un **univers directionnel indépendant**.

## 16.9. Destruction directionnelle et ID global

Lors d'une destruction directionnelle :

- l'ID global est invalidé,
- les IDs internes sont effacés,
- les chemins internes sont détruits,
- les paires normal/fantôme deviennent incohérentes,
- le Tore Privé est effacé.

L'ID global est **le premier élément détruit**.

## 16.10. Résumé du Chapitre 16

La cohérence directionnelle globale repose sur :

- l'ID global,
- les IDs internes,
- les chemins internes,
- les paires normal/fantôme,
- les fenêtres directionnelles,
- les déplacements fractals,
- la validation par la Forteresse de Verre.

L'ID global :

- est une structure fractale vivante,
- est un ID directionnel de **16 bits**,
- est produit par le **second segment du parcours fractal**,
- utilise une **valeur libre** non utilisée par les IDs de case,
- résume l'état du Tore Privé,
- stabilise la session,
- protège contre les attaques,
- est inviolable par conception,
- est détruit en cas d'anomalie.

Il constitue **la structure directionnelle centrale de TorusNet**.

# Chapitre 17 — Logique de Lecture des Tores (Privé et Public)

## 17.1. Vue d'ensemble

La lecture des tores est un élément fondamental du protocole TorusNet. Elle détermine :

- la progression fractale,
- la cohérence directionnelle,
- la reconstruction des fichiers,
- la reconstruction des clés,
- la mutation des IDs internes,
- la synchronisation Host  $\leftrightarrow$  Client  $\leftrightarrow$  Host.

Ce chapitre décrit la logique complète de lecture :

- du Tore Privé,
- du Tore Public,
- des déplacements exponentiels,
- des déplacements directionnels,
- du comportement circulaire du tore,
- de la lecture par colonnes complètes.

## 17.2. Lecture du Tore Privé

Le Tore Privé est un espace directionnel fermé. Sa lecture dépend du type de déplacement défini par la clé :

- déplacements exponentiels,
- déplacements directionnels (haut, bas, gauche, droite, diagonales).

Les deux modes sont complémentaires.

### 17.2.1. Lecture exponentielle : mode circulaire ligne-par-ligne

Lorsque le point de départ de la clé est lu et que les cases de déplacement donnent une valeur exponentielle, alors :

**Le Tore Privé est lu de façon circulaire, ligne par ligne.**

#### Règles de lecture circulaire

- Le tore est parcouru ligne par ligne, de gauche à droite.
- Quand la lecture atteint la fin d'une ligne, elle revient au début de la ligne suivante.
- Quand la lecture atteint la dernière ligne, elle revient à la première ligne.

- Le tore est un espace cyclique, sans bord, sans fin, sans limite.

### **Effets directionnels**

Ce mode permet :

- d'atteindre n'importe quelle case du tore,
- de créer des séquences fractales longues,
- d'assurer une cohérence directionnelle parfaite,
- d'éviter les collisions directionnelles.

### **17.2.2. Lecture directionnelle : haut, bas, gauche, droite, diagonales**

Lorsque les déplacements sont :

- haut,
- bas,
- gauche,
- droite,
- diagonales,

le Tore Privé se comporte comme un tore topologique réel.

### **Règle : wrap-around directionnel**

- Dépasser en haut → revenir en bas.
- Dépasser en bas → revenir en haut.
- Dépasser à gauche → arriver à droite.
- Dépasser à droite → arriver à gauche.
- Les diagonales combinent ces règles.

### **Origine du nom TorusNet**

**Ce principe de wrap-around est précisément ce qui définit un tore topologique. C'est la raison pour laquelle le protocole porte le nom *TorusNet* : un réseau dont la géométrie interne se comporte comme un tore, où chaque dépassement ramène naturellement à l'autre côté.**

### **Effets directionnels**

Ce mode permet :

- des déplacements locaux cohérents,
- des chemins internes stables,
- des inversions normal/fantôme précises,
- une progression fractale contrôlée.

## 17.3. Lecture du Tore Public

Le Tore Public est lu selon une logique différente. Les 30 bits de la clé directionnelle représentent **1000 déplacements fractals**, chacun donnant une valeur de colonne.

### Règle : lecture par colonnes complètes

Pour chaque déplacement fractal :

1. Les 30 bits sont interprétés par l'algorithme interne.
2. L'algorithme détermine **une colonne du Tore Public**.
3. Le programme lit **la colonne au complet**, de haut en bas.
4. Les 3 bits de chaque case sont utilisés pour :
  - déterminer les inversions normal/fantôme,
  - déterminer les chemins internes,
  - déterminer les mutations d'ID,
  - déterminer les déplacements fractals du Tore Privé.

### Effets directionnels

Ce mode permet :

- une extraction directionnelle massive,
- une cohérence fractale globale,
- une progression directionnelle profonde,
- une reconstruction de fichiers ou de clés.

## 17.4. Interaction avec les mises à jour directionnelles

### MAJ 1 — Inversion et mutation d'ID

La lecture du tore déclenche :

- l'inversion normal/fantôme,
- la mutation de l'ID de la case de départ.

### MAJ 2 — Suppression du renouvellement automatique

La lecture du tore n'est plus interrompue par un cycle horaire. Elle est continue, vivante, directionnelle.

### MAJ 3 — ID unique du tore

Si une clé contient l'ID unique du tore :

- le Client détecte un renouvellement directionnel,
- la lecture du tore bascule en mode de réinitialisation contrôlée.

## 17.5. Synthèse du Chapitre 17

La lecture des tores repose sur :

- un mode circulaire pour les déplacements exponentiels,
- un mode wrap-around pour les déplacements directionnels,
- une lecture par colonnes complètes pour le Tore Public,
- une cohérence directionnelle totale,
- une progression fractale vivante,
- une synchronisation Host ↔ Client ↔ Host.

Ce chapitre complète la mécanique fractale et stabilise la logique interne du protocole TorusNet.

# Chapitre 18 — Authentification DNS et HTTPS (Couche de Confiance Temporaire)

## 18.1. Vue d'ensemble

Avant que la géométrie directionnelle de TorusNet ne prenne le contrôle total de la session, une couche de confiance temporaire est nécessaire pour :

- identifier l'hôte demandé,
- éviter les redirections malveillantes,
- empêcher les attaques MITM classiques,
- sécuriser la transmission de la première clé directionnelle,
- valider que le client contacte bien le bon serveur.

Cette couche repose sur deux technologies de l'Internet classique :

- **DNS** (Domain Name System),
- **HTTPS** (certificat TLS).

Ces technologies ne font **pas partie** de la mécanique directionnelle. Elles servent uniquement de **sas d'entrée**, avant que TorusNet n'active :

- le Tore d'Authentification,
- le Tore Privé,
- le Tore Public,
- la Forteresse de Verre,
- les IDs internes,
- l'ID global.

Une fois la première clé validée, **DNS et HTTPS disparaissent complètement.**

## 18.2. Rôle du DNS dans TorusNet

Le DNS est utilisé pour une seule raison :

**Identifier l'hôte demandé avant d'entrer dans la géométrie directionnelle.**

### 18.2.1. Vérification du domaine

Lorsqu'un client contacte un serveur :

1. Le DNS résout le nom de domaine.
2. TorusNet vérifie que ce domaine appartient à la liste des hôtes de confiance.
3. Si le domaine est inconnu ou suspect :
  - la connexion passe en **mode confirmation utilisateur**,
  - aucune clé directionnelle n'est transmise.

### 18.2.2. Protection contre les redirections

Le DNS empêche :

- les redirections malveillantes,
- les faux domaines,
- les hôtes clonés,
- les détournements de trafic.

### 18.2.3. Rôle strictement temporaire

Une fois la première clé validée :

**Le DNS n'est plus utilisé. La géométrie directionnelle prend le relais.**

## 18.3. Rôle du HTTPS dans TorusNet

HTTPS est utilisé pour :

**sécuriser la transmission de la première clé directionnelle.**

### 18.3.1. Vérification du certificat

Le client vérifie :

- le certificat TLS,
- la validité du domaine,
- l'absence d'interception MITM classique.

Si le site n'a pas de HTTPS :

- la connexion passe en **mode confirmation utilisateur**,
- aucune clé directionnelle n'est envoyée.

### 18.3.2. Protection contre les attaques classiques

HTTPS protège contre :

- les interceptions,
- les injections,
- les manipulations de trafic,
- les attaques de downgrade.

### 18.3.3. Rôle strictement temporaire

Une fois la première clé transmise :

**HTTPS disparaît. La session devient 100 % directionnelle.**

## 18.4. Interaction DNS ↔ HTTPS ↔ Tore d'Authentification

La séquence complète est :

1. **DNS** identifie l'hôte.
2. **HTTPS** sécurise la première transmission.
3. **Le Tore d'Authentification** valide la première clé.
4. **La Forteresse de Verre** active la session directionnelle.
5. **Le Tore Privé** prend le contrôle.
6. **Le Tore Public** fournit la géométrie statique.
7. **DNS et HTTPS sont abandonnés.**

### 18.4.1. Cas de confiance totale

- DNS valide
- HTTPS valide → La première clé est transmise automatiquement.

### 18.4.2. Cas de confiance partielle

- DNS valide
- HTTPS absent → Confirmation utilisateur obligatoire.

### 18.4.3. Cas de suspicion

- DNS suspect
- HTTPS absent ou invalide → Connexion bloquée jusqu'à validation explicite.

## 18.5. Pourquoi TorusNet utilise encore DNS et HTTPS

TorusNet est une architecture directionnelle **post-Internet**. Cependant, l'écosystème actuel impose :

- l'usage de noms de domaine,
- l'usage de certificats TLS,

- l'usage de serveurs HTTP(S).

DNS et HTTPS servent donc de **pont** entre :

- l'Internet classique,
- l'Internet directionnel.

### 18.5.1. Objectif final : disparition totale

Dans les versions futures :

- le DNS sera remplacé par un **DNS directionnel**,
- HTTPS sera remplacé par une **validation directionnelle native**,
- les certificats seront remplacés par des **IDs directionnels**,
- les connexions seront entièrement géométriques.

## 18.6. Interaction avec les mises à jour directionnelles

### MAJ 1 — Inversion et mutation d'ID

Une fois la première clé validée :

- l'ID de la case de départ est muté,
- l'ID global est créé,
- DNS et HTTPS ne sont plus nécessaires.

### MAJ 2 — Suppression du renouvellement automatique

La session directionnelle devient :

- continue,
- vivante,
- autonome.

### MAJ 3 — ID unique du tore

Si une clé contient l'ID unique du tore :

- le client détecte un renouvellement directionnel,
- la session bascule en mode de réinitialisation contrôlée,
- DNS et HTTPS ne sont pas réutilisés.

## 18.7. Synthèse du Chapitre 18

L'authentification DNS et HTTPS :

- sert uniquement de sas d'entrée,
- protège la première clé directionnelle,
- empêche les attaques classiques,

- valide l'hôte demandé,
- disparaît dès que la géométrie directionnelle s'active.

TorusNet :

- utilise DNS et HTTPS par nécessité,
- mais ne dépend pas d'eux,
- les remplace immédiatement par la mécanique directionnelle,
- vise à les éliminer totalement dans les versions futures.

Ce chapitre complète la logique d'entrée du protocole TorusNet et prépare la transition vers une authentification 100 % directionnelle.

## Conclusion

Le développement de TorusNet et de Mondus marque une rupture profonde avec les fondations de l'Internet classique. En remplaçant les protocoles transmissifs par une architecture directionnelle, enracinée dans la géométrie des tores et l'identité matérielle des machines, le projet inaugure une nouvelle manière de concevoir la communication numérique : non transmissive, non dérivable, non clonable, et inviolable par conception.

Les mises à jour successives ont progressivement établi les bases d'un réseau autonome, capable de fonctionner sans cryptographie traditionnelle. Elles ont également défini les mécanismes de permutation directionnelle, de validation matérielle, de lecture orbitale, et de sécurité structurelle qui rendent chaque session unique, imprévisible et impossible à reproduire.

Mondus n'est pas une évolution de l'Internet : c'est une alternative totale, une réinvention complète de ce que peut être un réseau mondial. En s'appuyant sur TorusNet comme moteur directionnel, Mondus propose un modèle post-Internet où la sécurité n'est plus un ajout, mais une propriété naturelle de la structure elle-même.

À terme, que le projet soit adopté par les institutions ou publié en open-source, Mondus a vocation à devenir un standard global, capable de servir les besoins souverains, industriels, scientifiques et civils. Il ouvre la voie à une nouvelle ère technologique où les réseaux ne transmettent plus des données, mais des directions, et où la géométrie devient le fondement de la sécurité numérique.

Mondus représente ainsi la transition vers un futur où le réseau n'est plus seulement un outil, mais une architecture vivante, directionnelle et autonome — le premier véritable réseau post-Internet.